

PreOne Enterprise

Backend Technical Design

Backend Architecture — Enterprise Preschool Operating System

Document Version: 1.0

Status: Backend Architecture Freeze

Date: 2026-07-14

Product: PreOne — Enterprise Preschool Operating System

Predecessor: Vision v1.0, BRC v1.0, Master PRD v1.0, DDD v1.0, ADR Catalog, ERD v3.0, Prisma Schema v3.0, API Contract Catalog

Successor: Backend Implementation, Unit Tests, Integration Tests, Deployment, Production Release

Classification: Internal Engineering Reference

Prepared by: PreOne Architecture & Engineering Team

Table of Contents

1. Backend Overview.....	1
1.1 Purpose.....	1
1.2 Scope.....	2
1.3 Audience.....	3
1.4 Document Conventions.....	4
1.5 Related Documents.....	5
2. Technology Stack.....	5
2.1 Stack Pipeline.....	5
2.2 Component Catalog.....	8
2.3 Version Pinning Strategy.....	8
3. Solution Architecture.....	8
3.1 Layered Architecture.....	9
3.2 Request Lifecycle.....	11
3.3 Cross-Cutting Concerns.....	13
3.4 Real-Time Layer.....	14
4. Folder Structure.....	14
4.1 Top-Level Layout.....	15
4.2 Top-Level Directory Roles.....	17
4.3 Module Folders.....	18
5. Module Structure.....	18
5.1 Module Template (student/ example).....	18
5.2 Directory Responsibilities.....	22
5.3 Module Wiring (student.module.ts).....	22
6. Dependency Rules.....	25

6.1 Layer-to-Layer Rules.....	26
6.2 Cross-Module Communication.....	26
6.3 ESLint Configuration.....	27
7. Controllers.....	29
7.1 Controller Responsibilities.....	29
7.2 Controller Anti-Patterns.....	29
7.3 Example — POST /v1/students.....	30
8. DTOs.....	35
8.1 DTO Types.....	35
8.2 DTO Rules.....	36
8.3 Example — CreateStudentDto.....	37
9. Application Services.....	42
9.1 Responsibilities.....	43
9.2 Transaction Boundary Pattern.....	43
10. Domain Services.....	47
10.1 Responsibilities.....	47
10.2 Example — FeeCalculationService.....	48
11. Repositories.....	52
11.1 Repository Interface Contract.....	53
11.2 Repository Rules.....	53
11.3 Example — PrismaStudentRepository.....	54
12. CQRS.....	59
12.1 When to Use CQRS.....	59
12.2 Command Side.....	60
Command Catalog.....	61
12.3 Query Side.....	62
Query Catalog.....	63
13. Domain Events.....	63

13.1 Event Flow.....	63
13.2 Event Catalog.....	65
13.3 Event Schema Rules.....	65
14. Integration Events.....	67
14.1 Integration Event Catalog.....	67
14.2 Integration Patterns.....	67
14.3 Integration Rules.....	68
15. Background Jobs.....	69
15.1 BullMQ Queue Catalog.....	69
15.2 Retry Policy.....	70
15.3 Job Lifecycle.....	70
16. Cache Strategy.....	72
16.1 Cache Layers.....	72
16.2 Cache Patterns.....	72
16.3 Cache Invalidation Rules.....	73
16.4 Permission Cache Deep-Dive.....	74
17. Transactions.....	76
17.1 Transaction Patterns.....	76
17.2 Transaction Rules.....	77
17.3 Example — Admission Approval Saga.....	78
18. Validation.....	81
18.1 Validation Layers.....	82
18.2 Validation Flow.....	82
18.3 Validation Best Practices.....	84
19. Exception Handling.....	85
19.1 Exception Types.....	85
19.2 Standard Error Response.....	85
Response Field Reference.....	87

19.3 Global Exception Filter.....	87
20. Security.....	91
20.1 Security Controls Catalog.....	91
20.2 Authentication Flow.....	91
20.3 PII Protection.....	94
21. Multi-Tenant Strategy.....	94
21.1 Isolation Layers.....	94
21.2 Isolation Scope.....	94
21.3 PostgreSQL RLS Policy.....	95
22. Logging.....	97
22.1 Log Fields.....	97
22.2 Log Levels.....	97
22.3 Pino Configuration.....	98
23. Observability.....	101
23.1 Three Pillars.....	101
23.2 Health Checks.....	102
23.3 Alerting Strategy.....	102
24. Testing.....	103
24.1 Test Pyramid.....	103
24.2 Test Pyramid Diagram.....	103
24.3 Testing Rules.....	105
24.4 Example — Aggregate Unit Test.....	106
25. Performance.....	110
25.1 Performance Tactics.....	111
25.2 Service Level Objectives (SLOs).....	111
25.3 Connection Pooling.....	111
26. Coding Standards.....	113
26.1 Standards Catalog.....	114

26.2 Pre-commit Hooks.....	116
26.3 PR Review Checklist.....	118
27. Deployment Readiness.....	119
27.1 Pre-Deployment Checklist.....	120
27.2 Deployment Topology.....	120
27.3 CI/CD Pipeline.....	120
28. Glossary.....	123
29. Document Control.....	124
29.1 Version History.....	124
29.2 Approval Matrix.....	124
29.3 Review Cadence.....	124
29.4 Distribution List.....	125
29.5 Companion Documents.....	126
29.6 Deliverables.....	126

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

1. Backend Overview

1.1 Purpose

हा Document PreOne च्या Backend System ची complete technical blueprint प्रदान करतो. Backend हा PreOne platform चा core engine आहे जो 13 business domains, 530+ REST APIs, 14 NestJS modules, आणि शेकडो background jobs कोणत्याही वेळी orchestrate करतो. या document चा primary goal म्हणजे architecture, patterns, आणि standards ची एकच source of truth स्थापित करणे जी implementation, review, आणि onboarding साठी reference म्हणजे वापरली जाऊ शकेल.

Backend Technical Design (BTD) हा Vision, BRC, PRD, आणि DDD च्या वर build होतो — त्यांच्या business requirements ला concrete software architecture मध्ये translate करतो. DDD मध्ये define केलेले aggregates, entities, आणि bounded contexts या document मध्ये NestJS modules, Prisma schemas, आणि application services मध्ये implement होतात. प्रत्येक architectural choice ला ADR Catalog मध्ये documented decision आणि ERD v3.0 मध्ये physical schema backing आहे.

या document वरून पुढील production artifacts तयार होतील: Complete NestJS Folder Structure (14 domain modules), Controllers (~530 REST APIs), DTO Library, Application & Domain Services, CQRS Commands / Queries / Handlers, Domain Event Catalog, BullMQ Job Definitions, Redis Cache Design, Transaction Strategy, Exception & Validation Framework, Logging & Observability Standards, Testing Strategy, आणि Backend Implementation Guidelines.

1.2 Scope

हा document फक्त Backend System च्या architecture आणि implementation guidelines cover करतो. खालील explicitly out of scope आहेत:

- Frontend / Mobile Architecture — separate frontend design document मध्ये cover होते.
- Infrastructure as Code (Terraform / Pulumi) — DevOps runbook मध्ये documented आहे.
- Third-party SDK integration details — per-provider spec sheets (SMS, WhatsApp, Payment gateways) reference केले आहेत.
- Mobile Push (FCM / APNs) cert management — operations runbook मध्ये.
- Data migration scripts from legacy systems — separate migration runbook.

1.3 Audience

हा document खालील audiences साठी लिखित आहे:

- **Backend Engineers:** Primary reference — module structure, patterns, coding standards, testing strategy implementation साठी.
- **Tech Leads / Architects:** Architecture enforcement — dependency rules, CQRS applicability, transaction boundaries review साठी.
- **QA Engineers:** Testing pyramid understanding, integration test patterns, आणि SLO targets साठी.
- **DevOps / SRE:** Deployment readiness checklist, observability stack, health checks, आणि scaling patterns साठी.
- **AI Code Assistants:** Each section structured as a prompt-able block; AI tools च्या context मध्ये दिल्यावर accurate NestJS code generation होते.
- **New Hires:** Onboarding reference — first 2 weeks मध्ये वाचून backend mental model तयार होतो.

1.4 Document Conventions

- Module names: kebab-case (student, attendance, identity).
- Class names: PascalCase (StudentAggregate, CreateStudentCommand).
- File names: kebab-case (student.aggregate.ts, create-student.dto.ts).
- Database tables: snake_case (students, admission_applications).
- REST endpoints: kebab-case plural (/v1/students, /v1/attendance-records).
- Code blocks: monospace font (Consolas); tree diagrams use ^L – characters.
- Bilingual content: Marathi business context + English technical terms preserved exactly as user provided.

1.5 Related Documents

PreOne document series मध्ये BTD चे specific position आहे. खालील documents BTD चे predecessors आहेत:

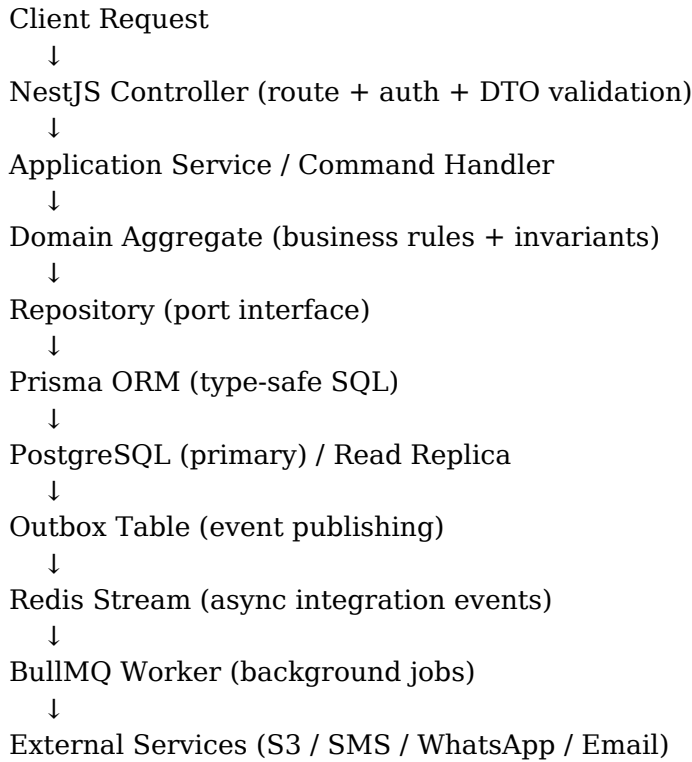
Document	Version	Status	Relationship to BTD
Vision Document	v1.0	Vision Freeze	Strategic intent — BTD च्या non-functional requirements (scale, compliance) Vision शी align
BRC	v1.0	Business Freeze	176 rules — BTD च्या Domain Events + Specifications BRC चे implementation
Master PRD	v1.0	Product Freeze	Functional requirements — BTD चे API endpoints PRD च्या FRs चे direct mapping
DDD	v1.0	Architecture Freeze	Domain model — BTD च्या NestJS modules DDD च्या bounded contexts चे implementation
ADR Catalog	v1.0	Architecture Freeze	Decisions — BTD प्रत्येक architectural choice ला ADR reference करतो
ERD v3.0	v3.0	Schema Freeze	Physical schema — BTD च्या Prisma models ERD चे direct translation
Prisma Schema v3.0	v3.0	Schema Freeze	Code-first schema — BTD च्या repositories चे data access layer
API Contract Catalog	v1.0	API Freeze	Endpoint contracts — BTD च्या controllers चे spec

2. Technology Stack

PreOne backend चा technology stack enterprise-grade reliability, developer productivity, आणि AI-assisted development optimize करण्यासाठी choose केला आहे. प्रत्येक choice ला ADR (Architecture Decision Record) backing आहे जो trade-offs documented करतो. Stack चा core philosophy म्हणजे type safety end-to-end (TypeScript + Prisma + Zod), modularity (NestJS DI), आणि operational maturity (BullMQ + OpenTelemetry + Pino) ।

2.1 Stack Pipeline

Request चा flow stack च्या प्रत्येक layer through जातो. ही pipeline request lifecycle समजण्यासाठी critical आहे:



2.2 Component Catalog

खालील table सर्व stack components ची authoritative list आहे. नवीन component introduce करताना या table मध्ये entry जोडावी लागेल आणि corresponding ADR draft करावा लागेल:

Component	Version	Role	Highlights	Why Chosen
NestJS	v11.x	Application Framework	Modular DI, decorators, microservices, WebSocket gateway, scheduling, OpenAPI via @nestjs/swagger	Production-grade Angular-style DI; ecosystem mature; squad familiarity
TypeScript	v5.4+	Language	Strict mode, path aliases, decorators, const assertions, satisfies operator	Type safety at compile time; DDD-friendly with branded types & value objects
Prisma ORM	v6.x	ORM / Data Access	Schema-first, type-safe client, migrations, multi-schema, read-replica routing (client extensions)	Type safety end-to-end; migration history; excellent Postgres support
PostgreSQL	v16	Primary Database	RLS, partial indexes, JSONB, generated columns, pgcrypto, pgvector (for AI), logical replication	ACID compliance; RLS for tenant isolation; pgvector for embeddings
Redis	v7.2	Cache + Queue Backend	Permission cache, menu cache, BullMQ backend, rate-limit store, pub/sub	Sub-ms reads; supports BullMQ persistence; Redis Streams for events

Component	Version	Role	Highlights	Why Chosen
BullMQ	v5.x	Background Job Queue	Repeatable jobs, delayed jobs, priorities, rate-limited queues, DLQ, flow producer	Battle-tested on Redis; observability via Bull Board; sandboxed processors
JWT (jose)	v5.x	Authentication Tokens	Access tokens (15 min) + refresh tokens (30 d, rotated), RS256 signed	Stateless verification; horizontal scaling friendly
WebSocket (Socket.IO)	v4.x	Real-Time Layer	Adaptive transport, rooms per tenant+branch, sticky-session via Redis adapter	Bi-directional push for chat, attendance live, dashboards
AWS S3	—	Object Storage	Pre-signed uploads, server-side encryption (SSE-KMS), lifecycle to Glacier, versioning	Durable file storage; lifecycle to Glacier for archives; KMS encryption
Zod / class-validator	—	Validation	DTO validation pipeline; schema composition; custom decorators	Pipe integration; fail-fast on invalid input
Pino	v9.x	Logging	JSON structured logs, child loggers, redact PII, async transport	Low overhead; structured for Loki/ELK; correlation by traceId

Component	Version	Role	Highlights	Why Chosen
OpenTelemetry	v1.x	Tracing / Metrics	Auto-instrumentation for HTTP/PG/Redis, custom spans, OTLP exporter	Vendor-neutral; spans correlated to logs
Vitest	v1.x	Unit Test Runner	Native ESM, snapshot, in-source testing, coverage via v8	Fast startup; Jest-compatible API; TS native
Testcontainers	v10.x	Integration Tests	Real Postgres + Redis containers per test run	Real DB semantics; no mocked Prisma drift
ESLint + Prettier	v9.x	Linting & Format	Import boundaries (eslint-plugin-import), functional patterns, no-magic-numbers	Enforce dependency rules; format consistency
Husky + lint-staged	—	Pre-commit Hooks	Lint, format, type-check on staged files	Catch issues before push; block broken commits
Compodoc	—	Module Documentation	Auto-generated module diagrams from NestJS sources	Living architecture docs
Helmet + compression	—	HTTP Hardening	CSP, HSTS, no-sniff, gzip/brotli	Standard OWASP headers; payload size reduction

2.3 Version Pinning Strategy

Production stability साठी सर्व dependencies pinned आहेत — caret ranges (~) फक्त patch updates allow करतात, minor/major bumps ला explicit review लागतो. Lockfile (bun.lock)

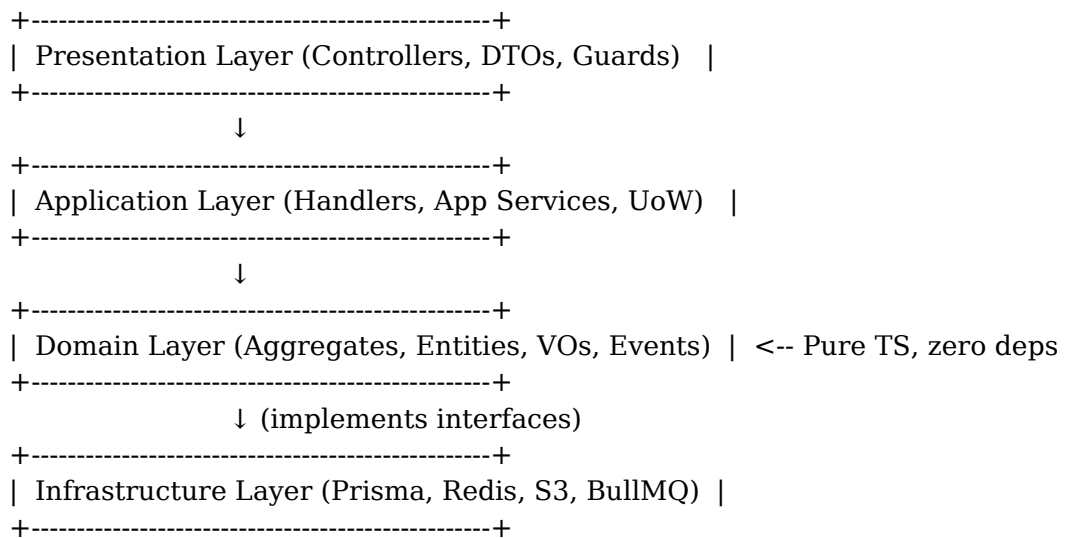
committed आहे आणि Renovate Bot weekly PRs raise करतो for updates. Critical security patches emergency channel through release होतात — on-call engineer च्या approval नंतर 4 hours मध्ये deploy.

3. Solution Architecture

PreOne चा backend हा modular monolith architecture follow करतो — single deployable NestJS application ज्यात 14 bounded contexts (modules) clean boundaries सह co-exist करतात. हा decision ADR-003 मध्ये documented आहे: microservices चे operational overhead (service discovery, distributed tracing, network failures) टाळण्यासाठी modular monolith प्रथम phase मध्ये choose केला आहे. जेव्हा specific module ची scale demand वाढेल (e.g., AI inference, Report generation), तेव्हा त्याला separate microservice मध्ये extract केले जाईल — DDD च्या bounded context boundaries हे extraction smooth करतील.

3.1 Layered Architecture

Backend चे चार layers आहेत — Presentation, Application, Domain, Infrastructure. हे layers DDD च्या standard pattern follow करतात आणि dependency rule enforce करतात की dependencies फक्त inward point करतात:



3.2 Request Lifecycle

प्रत्येक HTTP request खालील stages through जातो. ही traceability observability मध्ये critical आहे — कोणताही bottleneck trace ID ने identify केला जाऊ शकतो:

- Middleware: traceId generation, request logging, tenantId extraction.
- Guard (JwtAuthGuard): token verification, user context population.

- Guard (PermissionsGuard): RBAC check via Casbin policies.
- Pipe (ValidationPipe): DTO class-validator + Zod schema validation.
- Interceptor (LoggingInterceptor): request/response logging with duration.
- Interceptor (CacheInterceptor): Redis cache lookup for GET requests.
- Controller: route handler invocation.
- Handler / Application Service: business orchestration, UoW, event publishing.
- Domain Aggregate: invariant enforcement, state mutation.
- Repository (Prisma): SQL execution, optimistic lock check.
- Outbox: event persistence in same transaction.
- Response: ResponseDto envelope, status code, headers.
- Exception Filter (on error): standardised error response, traceId correlation.

3.3 Cross-Cutting Concerns

काही concerns सर्व modules ला apply होतात — ते global modules मध्ये implemented आहेत जे AppModule मध्ये registered आहेत. यामुळे per-module boilerplate कमी होते आणि consistency सुनिश्चित होते:

- **Logging:** Pino global logger with traceId, userId, tenantId auto-injected via AsyncLocalStorage.
- **Validation:** Global ValidationPipe with Zod + class-validator, whitelist + forbidNonWhitelisted.
- **Exception Handling:** Global ExceptionFilter catching all errors, mapping to standard response shape.
- **Auth:** Global JwtAuthGuard with @Public() decorator opt-out for login/health.
- **RBAC:** Global PermissionsGuard backed by Casbin enforcer (cached in Redis).
- **Rate Limiting:** Global ThrottlerGuard — per-IP and per-user limits.
- **CORS / Helmet:** Standard OWASP headers + CORS allowlist.
- **OpenTelemetry:** Auto-instrumentation for HTTP, PG, Redis, BullMQ — spans auto-created.
- **Trace Context:** traceId propagated via W3C Trace Context header (traceparent).

3.4 Real-Time Layer

WebSocket (Socket.IO) layer real-time features साठी आहे — parent chat, live attendance dashboard, instant notifications. Multi-instance deployment मध्ये Redis adapter वापरून cross-instance broadcast केले जाते. Connection authentication JWT ने होते; room join करताना tenantId + branchId ने namespace isolate केले जाते ज्यामुळे cross-tenant data leak होऊ शकत नाही.

4. Folder Structure

Project folder structure हे developer navigation चा foundation आहे. PreOne मध्ये strict folder convention आहे जो ESLint import boundaries ने enforce केला जातो — कोणतीही file wrong folder मध्ये place केल्यास lint error येतो आणि build fail होतो. ही discipline onboarding friction कमी करते आणि code discoverability वाढवते.

4.1 Top-Level Layout

src/ folder च्या खालील top-level directories आहेत. प्रत्येक directory ची specific responsibility आहे आणि cross-directory imports नियमित आहेत:

```
src/
├─ app/           # Bootstrap, global modules, filters, pipes
├─ common/        # Shared kernel (errors, types, utils)
├─ config/        # Typed config loaders
├─ infrastructure/ # Cross-module infra (Prisma, Redis, S3)
├─ shared/        # DDD kernel (base entity, VO, spec)
├─ modules/       # Business modules (bounded contexts)
│   └─ identity/
│   └─ crm/
│   └─ admissions/
│   └─ student/
│   └─ academics/
│   └─ attendance/
│   └─ communication/
│   └─ finance/
│   └─ inventory/
│   └─ hr/
│   └─ administration/
│   └─ reports/
│   └─ settings/
│   └─ platform/
```

4.2 Top-Level Directory Roles

Directory	Responsibility	Notes
app/	Application bootstrap, global modules, filters, pipes, interceptors, guards	Crown of the process; one-time wiring
common/	Shared kernel — base classes, error codes, result types, decorators, utilities, types	Cross-cutting, dependency-free
config/	Typed config loaders (env, secrets, tenant config, feature flags)	Loaded once at bootstrap
infrastructure/	Cross-module infra — Prisma client, Redis client, S3 client, EventBus, Mailer, BullMQ registrars	Concrete adapters; domain sees only interfaces
shared/	Shared domain primitives — UoW base, base entity, base aggregate, value object base, specification base	DDD kernel reused by all modules
modules/	Business modules (one folder per bounded context)	Where the actual product lives

4.3 Module Folders

modules/ directory मध्ये 14 business domains चे एक folder आहे. प्रत्येक folder स्वतःचे sub-structure follow करतो (Section 5 मध्ये detail). नवीन module add करताना AppModule मध्ये registration + ESLint boundary rule update + Compodoc diagram regenerate लागते. Module list खालीलप्रमाणे आहे:

#	Module	Primary Domain	Approximate API Count
1	identity	Users, Roles, Permissions, Tenants, Branches	~70
2	crm	Leads, Campaigns, Conversions	~40

#	Module	Primary Domain	Approximate API Count
3	admissions	Applications, Counselling, Approvals	~50
4	student	Student Lifecycle, Profiles, Guardians	~55
5	academics	Curriculum, Observations, Report Cards	~60
6	attendance	Daily Attendance, Arrival, Pickup	~35
7	communication	Announcements, Chat, Notifications	~50
8	finance	Fees, Invoices, Payments, Ledger, GST	~80
9	inventory	Items, Stock, PR, PO, GRN	~45
10	hr	Staff, Payroll, Leave, Attendance	~45
11	administration	Assets, Maintenance, Visitors	~25
12	reports	Cross-domain Reports, Analytics	~30
13	settings	Academic Years, Calendars, Configs	~20
14	platform	Subscriptions, Billing, Feature Flags	~25

5. Module Structure

PreOne चा प्रत्येक business domain identical folder template follow करतो. हे consistency onboarding friction कमी करते — developer एक module शिकल्यावर दुसरा module navigate करणे intuitive होते. Template DDD patterns (aggregates, repositories, domain events) आणि NestJS conventions (module, controllers, providers) चा hybrid आहे.

5.1 Module Template (student/ example)

खालील tree student/ module चे complete structure दाखवते. हेच pattern सर्व 14 modules साठी apply होते — फक्त folder name बदलते:

student/	
└ controllers/	# HTTP controllers
└ application/	
└ commands/	# CQRS command objects
└ queries/	# CQRS query objects
└ handlers/	# Command & query handlers
└ dto/	# Request/Response DTOs
└ mappers/	# DTO ↔ Domain ↔ Persistence
└ services/	# Application services (orchestration)
└ domain/	
└ aggregates/	# Aggregate roots + child entities
└ entities/	# Entities inside aggregates
└ value-objects/	# Immutable VOs (Address, Money)
└ repositories/	# Repository interfaces (ports)
└ services/	# Domain services (pure rules)
└ events/	# Domain event classes
└ policies/	# Business policies
└ specifications/	# Spec pattern (predicates)
└ infrastructure/	
└ prisma/	# Prisma schema slice + client
└ repositories/	# Prisma-backed implementations
└ cache/	# Redis cache wrappers
└ events/	# Event bus adapters
└ jobs/	# BullMQ producers + processors
└ presentation/	# Presenters (view-models)
└ student.module.ts	# NestJS module wiring

5.2 Directory Responsibilities

प्रत्येक sub-directory ची specific responsibility आहे. खालील table सर्व directories चे purpose + boundaries document करते:

Directory	Responsibility	Critical Rule
controllers/	HTTP controllers — versioned, route mapped, Swagger tagged	Controllers MUST NOT contain business logic; only DTO validation + dispatch
application/commands/	CQRS command objects (immutable, validated)	e.g., CreateStudentCommand, PromoteStudentCommand

Directory	Responsibility	Critical Rule
application/queries/	CQRS query objects for read side	e.g., GetStudentByIdQuery, SearchStudentsQuery
application/handlers/	Command & query handlers — orchestrate aggregate + UoW + events	Single handler per command/query; Idempotent via idempotency-key
application/dto/	Request/Response DTOs + mappers	Zod schemas + class-validator decorators + Swagger metadata
application/mappers/	DTO ↔ Domain ↔ Persistence mappers	Three-way mapping; never leak Prisma types to controllers
application/services/	Application services — multi-aggregate orchestration, transactional	Coordinate repositories; publish integration events
domain/aggregates/	Aggregate roots + child entities	Pure TS; zero infra deps; enforce invariants
domain/entities/	Entities inside aggregates (identity within parent)	Only mutate via aggregate root
domain/value-objects/	Immutable value objects (Address, Money, Age, TimeWindow)	Equals by value; throw on invalid construction
domain/repositories/	Repository interfaces (port)	Implemented in infrastructure/; mockable in tests
domain/services/	Domain services — pure business rules involving multiple aggregates	No state; no IO
domain/events/	Domain event classes	Past-tense; serialisable; versioned schema
domain/policies/	Business policies (e.g., RefundPolicy, LateFeePolicy)	Decision tables; pure functions
domain/specifications/	Specification pattern for complex predicate logic	Composable with .and(), .or(), .not()

Directory	Responsibility	Critical Rule
infrastructure/prisma/	Prisma schema slice + client wiring	Per-module Prisma schema fragment
infrastructure/repositories/	Concrete Prisma-backed repository implementations	Map Prisma rows ↔ domain aggregates
infrastructure/cache/	Redis cache wrappers (read-through, write-through)	TTL + invalidation versioned by cache key prefix
infrastructure/events/	Event bus adapters — in-memory (sync) + Redis Streams (async)	Outbox pattern for cross-aggregate reliability
infrastructure/jobs/	BullMQ job producers + processors for this module	Worker isolation; sandboxed processors
presentation/	Presenters — format domain responses for HTTP/WebSocket views	View-model layer; keeps DTOs clean
student.module.ts	NestJS module wiring — providers, controllers, imports, exports	Single source of DI graph

5.3 Module Wiring (student.module.ts)

प्रत्येक module ची .module.ts file NestJS DI container चा configuration point आहे. ही file imports, providers, controllers, exports declare करते. हे wiring ADR-007 मध्ये defined pattern follow करते — providers list मध्ये interfaces bind केल्या जातात concrete implementations सह, ज्यामुळे testing मध्ये swap करणे सोपे होते:

```
@Module({
  imports: [
    PrismaModule,
    RedisModule,
    EventBusModule,
    BullmqModule.registerQueue({ name: 'student-jobs' }),
  ],
  controllers: [StudentsController, StudentProfileController],
  providers: [
    // Application
    CreateStudentHandler,
    UpdateStudentHandler,
    GetStudentHandler,
    SearchStudentsHandler,
    StudentMapper,
    // Domain
```

```

    { provide: 'StudentRepository', useClass: PrismaStudentRepository },
    StudentFactory,
    AgeEligibilitySpecification,
    // Infrastructure
    StudentCache,
    { provide: 'StudentEventBus', useClass: RedisStreamEventBus },
  ],
  exports: ['StudentRepository', 'StudentEventBus'],
})
export class StudentModule {}

```

6. Dependency Rules

Dependency rules हे architecture चे enforcement mechanism आहेत. हे rules ESLint plugin (eslint-plugin-import) ने code level पर enforce केले जातात — violation साठी lint error येतो आणि build fail होतो. यामुळे architecture drift detect होतो जेव्हा developer accidentally wrong layer import करतो.

6.1 Layer-to-Layer Rules

खालील table सर्व possible layer-to-layer import combinations ची status document करते. 'Allowed' किंवा 'Forbidden' tags strict enforcement सूचित करतात:

From → To	Status	Notes
Presentation → Application	Allowed	Controllers call application services / command handlers
Presentation → Domain	Forbidden	Controllers must never touch aggregates/entities directly
Presentation → Infrastructure	Forbidden	Controllers must never import Prisma/Redis/S3
Application → Domain	Allowed	Handlers orchestrate aggregates via repository ports
Application → Infrastructure	Allowed (via interfaces)	Repositories injected as abstractions; never concrete class
Application → Presentation	Forbidden	No back-flow into DTOs/controllers
Domain → Application	Forbidden	Domain knows nothing about use cases

From → To	Status	Notes
Domain → Infrastructure	Forbidden	Domain is pure TypeScript; zero NestJS/Prisma imports
Domain → Domain	Allowed (same module)	Cross-module imports forbidden — use integration events
Infrastructure → Domain	Allowed (implements interfaces)	Repos implement domain-side ports
Infrastructure → Application	Forbidden	Infra must not know about handlers
Cross-module direct imports	Forbidden	Modules talk via ACL (Anti-Corruption Layer) or events

6.2 Cross-Module Communication

Modules एकमेकांशी direct imports करू शकत नाहीत — हे strict rule आहे. Cross-module communication खालील patterns through होते:

- **Integration Events (Async):** Redis Stream पर event publish करून consumer module subscribe करतो. Most common pattern — eventual consistency acceptable.
- **HTTP Call (Sync):** Same-process module-to-module call internal HTTP endpoint वर. Used when immediate response required (e.g., Admissions → Identity for student ID).
- **ACL (Anti-Corruption Layer):** Calling module त्याच्या own context मध्ये translation layer ठेवते जो external module चे model translate करतो. Prevents model pollution.
- **Shared Kernel:** common/ आणि shared/ directories मधील code सर्व modules import करू शकतात — base classes, error types, utilities.
- **Read Model Projection:** Module दुसऱ्या module चा read-only data ची copy ठेवते via event subscription (CQRS pattern). Avoids cross-module queries.

6.3 ESLint Configuration

Import boundary rules ESLint config मध्ये defined आहेत. खालील snippet rules चा essence दाखवतो — full config repo मध्ये .eslintrc.js मध्ये आहे:

```
'import/no-restricted-paths': ['error', {
  zones: [
    // Controllers cannot import domain or infra
```

```

    { target: './src/modules/**/controllers', from: './src/modules/**/domain' },
    { target: './src/modules/**/controllers', from:
'./src/modules/**/infrastructure' },
    // Domain cannot import anything external
    { target: './src/modules/**/domain', from: './src/modules/**/application' },
    { target: './src/modules/**/domain', from: './src/modules/**/infrastructure' },
    { target: './src/modules/**/domain', from: './node_modules' },
    // Cross-module imports forbidden
    { target: './src/modules/identity', from: './src/modules/finance' },
    // ... one entry per module pair
  ],
}],

```

7. Controllers

Controllers फक्त HTTP layer handle करतील. Controllers मध्ये business logic नसणार — हा DDD coding guideline चा main नियम आहे. Controller चा responsibility सीमित आहे: request receive करणे, validate करणे, application layer कडे delegate करणे, response format करून return करणे. Business decisions handler किंवा domain aggregate मध्ये होतात.

7.1 Controller Responsibilities

खालील table प्रत्येक controller responsibility चे detail देते. हे responsibilities ADR-008 मध्ये defined आहेत आणि PR review मध्ये verify केले जातात:

Responsibility	Mechanism	Notes
Route Mapping	@Controller('v1/students'), @Get(':id'), @Post() — versioned URLs	Path includes /v1/ for forward compatibility
Authentication	@UseGuards(JwtAuthGuard) — verifies JWT, populates req.user	Global guard; opt-out via @Public() for login/health
Authorization	@Permissions('students:read) — RBAC data scope check	Custom decorator backed by Casbin policies
DTO Validation	@Body() dto: CreateStudentDto — class- validator + Zod pipe	Fail fast on schema violation; 422 response
Response Formatting	Returns ResponseDto<T> envelope	Consistent { success, data, traceld } shape
OpenAPI Metadata	@ApiOperation, @ApiResponse, @ApiTags	Swagger UI auto-generated

Responsibility	Mechanism	Notes
Idempotency	@Idempotent() decorator for POST/PUT	Key from header or body hash; 24h TTL in Redis
Rate Limiting	@Throttle(20, 60) per route	Bullhorn or per-IP + per-user

7.2 Controller Anti-Patterns

खालील anti-patterns PR review मध्ये block केले जातील. कोणताही controller यापैकी कोणता pattern exhibit करत असेल तर PR reject होईल आणि refactor request केला जाईल:

- Business logic in controller (use handler instead)
- Direct Prisma calls in controller (use application service)
- Returning domain entities from controller (return DTOs only)
- Conditional logic beyond input validation (move to handler)
- Catching business exceptions (let global filter handle them)
- Multiple responsibilities per route (split into separate endpoints)
- Skipping Swagger metadata (every route must be documented)
- Using req.body raw without DTO (always validate)

7.3 Example — POST /v1/students

खालील example सर्व layers कसे coordinate करतात ते दाखवते. Controller फक्त orchestration करतो — actual business logic aggregate मध्ये, persistence repository मध्ये, event publishing application service मध्ये:

```
@Controller('v1/students')
@ApiTags('students')
@UseGuards(JwtAuthGuard, PermissionsGuard)
export class StudentsController {
  constructor(
    private readonly commandBus: CommandBus,
    private readonly queryBus: QueryBus,
  ) {}

  @Post()
  @Permissions('students:create')
  @Idempotent({ ttl: 86400 })
  @ApiOperation({ summary: 'Create new student' })
  @ApiResponse({ status: 201, type: StudentResponseDto })
  async create(
    @Body() dto: CreateStudentDto,
```

```

    @ReqUser() user: AuthenticatedUser,
  ): Promise<ResponseDto<StudentResponseDto>> {
    const command = new CreateStudentCommand(dto, user);
    const student = await this.commandBus.execute(command);
    return ResponseDto.success(StudentMapper.toResponse(student));
  }
}

// Flow:
// POST /students
//   ↓
// CreateStudentCommand (validated DTO)
//   ↓
// CreateStudentHandler (orchestration + UoW)
//   ↓
// StudentAggregate (invariants enforced)
//   ↓
// StudentRepository.save() (Prisma persistence)
//   ↓
// StudentCreatedEvent (outbox → Redis Stream)

```

8. DTOs

Data Transfer Objects (DTOs) ही process / network boundaries पर data carry करण्याचा mechanism आहे. PreOne मध्ये DTOs ची strict typing आणि validation असते — कोणताही untyped payload accept होत नाही. हे discipline type safety end-to-end सुनिश्चित करते आणि runtime errors compile-time ला move होतात.

8.1 DTO Types

PreOne मध्ये 8 primary DTO types आहेत. प्रत्येक type ची specific purpose आहे आणि त्याच्या वर खास decorators apply होतात:

DTO Class	Type	Endpoint	Purpose
CreateStudentDto	Request	POST /v1/students	All required + optional fields, validation decorators, Swagger metadata
UpdateStudentDto	Request	PATCH /v1/students/:id	PartialType(CreateStudentDto) — all fields optional

DTO Class	Type	Endpoint	Purpose
StudentResponseDto	Response	GET /v1/students/:id	Sanitised output; never includes PII like Aadhaar
StudentListItemDto	Response	GET /v1/students	Lightweight fields for list view; pagination metadata
StudentFilterDto	Query	GET /v1/students?filter[...]	Filterable fields, sort spec, pagination params
StudentSearchDto	Query	GET /v1/students/search	Full-text search query, fuzzy match, suggestions
BulkCreateStudentDto	Request	POST /v1/students/bulk	Array of CreateStudentDto + idempotency key per item
StudentExportDto	Query	GET /v1/students/export	Filter + format (xlsx/csv/pdf) + columns spec

8.2 DTO Rules

खालील rules सर्व DTOs साठी mandatory आहेत. PR review मध्ये verify केले जाते की नवीन DTO यापैकी प्रत्येक rule follow करत आहे:

- Every DTO must extend a base DTO with traceId + correlationId
- Request DTOs use class-validator decorators (@IsString, @IsEmail, @MinLength, etc.)
- Request DTOs also define Zod schema for runtime validation (double-layer)
- Response DTOs use @ApiProperty with example values for Swagger
- PII fields in response DTOs masked via @Mask('***-**-****') decorator
- Never expose internal domain entity types in DTOs — always map via Presenter
- DTOs are immutable (readonly fields) — modification is anti-pattern
- Bulk operations use array DTO with maxItems=500 enforced
- Date fields use ISO-8601 string (not Date object) for portability

- Money fields stored as integer minor units (paise) with currency code
- Pagination cursor is opaque base64 — never decoded client-side
- Every breaking DTO change requires DTO version bump (V2 suffix)

8.3 Example — CreateStudentDto

खालील example complete DTO definition दाखवते — class-validator decorators, Zod schema, Swagger metadata एकाच file मध्ये. हे triple validation belt type safety maximum करते:

```
export class CreateStudentDto {
  @ApiProperty({ example: 'Aarav', description: 'Student first name' })
  @IsString() @MinLength(1) @MaxLength(50)
  @Matches(/^[p{L}]s\.-]+$/u, { message: 'Invalid name format' })
  readonly firstName!: string;

  @ApiProperty({ example: 'Sharma', description: 'Student last name' })
  @IsString() @MinLength(1) @MaxLength(50)
  readonly lastName!: string;

  @ApiProperty({ example: '2019-05-15', description: 'Date of birth (ISO-8601)' })
  @IsDateString() @IsBeforeThan('today')
  readonly dateOfBirth!: string;

  @ApiProperty({ example: 'MALE', enum: Gender })
  @IsEnum(Gender)
  readonly gender!: Gender;

  @ApiProperty({ type: () => AddressDto })
  @ValidateNested() @Type(() => AddressDto)
  readonly address!: AddressDto;

  @ApiProperty({ type: () => [GuardianDto], minItems: 1 })
  @ValidateNested({ each: true }) @Type(() => GuardianDto)
  @ArrayMinSize(1)
  readonly guardians!: GuardianDto[];
}

// Zod schema for runtime validation (double-layer)
export const CreateStudentSchema = z.object({
  firstName: z.string().min(1).max(50),
  lastName: z.string().min(1).max(50),
  dateOfBirth: z.string().datetime(),
  gender: z.nativeEnum(Gender),
  address: AddressSchema,
  guardians: z.array(GuardianSchema).min(1),
});
```

9. Application Services

Application Services हे workflow orchestration चे hub आहेत. ते transaction boundaries define करतात, aggregates coordinate करतात, आणि integration events publish करतात. Domain logic त्यांच्या आत नसते — ते फक्त glue code आहे जे domain layer कडे delegate करते. यामुळे domain layer pure राहते आणि application concerns (transactions, events) isolated होतात.

9.1 Responsibilities

Responsibility	Mechanism	Example
Workflow Orchestration	Coordinate multiple aggregates in a single transaction	EnrollStudentService → Student + Enrollment + Invoice + FeePlan
Transaction Boundary	Define UoW scope, commit/rollback	Single aggregate = single transaction; multi-aggregate via saga
Event Publishing	Dispatch domain events after commit	Outbox pattern — events written to DB, then published async
Cross-Domain Coordination	Call ACL of another bounded context	Finance → Identity ACL for user lookup
Authorization Decision	Check permissions before invoking domain	@PreAuthorize decorator or explicit check
Caching Decision	Read-through cache for read-heavy paths	Idempotent reads only; cache key = method signature + args
Idempotency Enforcement	Check idempotency-key in Redis	Return cached response if key exists; else compute + cache
Audit Trail	Persist who did what when	Audit log entry per command, immutable

9.2 Transaction Boundary Pattern

Application service हा transaction boundary आहे. प्रत्येक public method एका UoW मध्ये wrap केलेला असतो — commit सर्व operations successful असल्यावर होतो, rollback कोणतीही exception आल्यास होतो. यामुळे partial state updates prevent होतात:

```

@Injectable()
export class EnrollStudentService {
  constructor(
    @Inject('StudentRepository') private students: StudentRepository,
    @Inject('EnrollmentRepository') private enrollments: EnrollmentRepository,
    @Inject('InvoiceRepository') private invoices: InvoiceRepository,
    private readonly uow: UnitOfWork,
    private readonly eventBus: EventBus,
  ) {}

  async execute(cmd: EnrollStudentCommand): Promise<Enrollment> {
    return this.uow.execute(async (txn) => {
      // 1. Fetch aggregates
      const student = await this.students.findByIdOrThrow(cmd.studentId, txn);
      const section = await this.sections.findByIdOrThrow(cmd.sectionId, txn);

      // 2. Domain operation (pure business logic)
      const enrollment = student.enroll(section, cmd.academicYearId);

      // 3. Generate invoice (domain event raised inside)
      const invoice = student.generateInvoiceForTerm(cmd.feePlanId);

      // 4. Persist
      await this.students.save(student, txn);
      await this.enrollments.save(enrollment, txn);
      await this.invoices.save(invoice, txn);

      // 5. Publish events (via outbox - same txn)
      await this.eventBus.publishAll(student.pullEvents(), txn);

      return enrollment;
    });
  }
}

```

10. Domain Services

Domain Services हे pure business rules encapsulate करतात जे single aggregate मध्ये fit होत नाहीत. उदा. FeeCalculationService हे FeePlan, Concession, LateFee या तीन aggregates चा combination वापरून final invoice amount calculate करतो. Domain Service ला state नसते, IO नसते — फक्त pure functions. यामुळे ते 100% unit testable असतात.

10.1 Responsibilities

Responsibility	Description	Example
Pure Business Rules	Encapsulate logic that doesn't fit a single aggregate	FeeCalculationService — combines FeePlan + Concession + LateFee
Aggregate Coordination	Coordinate multiple aggregates without persistence	TransferStudentService moves student between sections
Policy Execution	Apply business policies	RefundPolicy decides refund amount given cancellation date
Specification Composition	Chain specifications for complex predicates	EligibilitySpec = AgeSpec.and(DocumentSpec).and(PaymentSpec)
Invariant Enforcement	Validate cross-entity invariants	No two students same roll number in same section
State Transitions	Aggregate state machine decisions	AdmissionAggregate.advanceState() checks allowed transitions

10.2 Example — FeeCalculationService

खालील example domain service चे pure nature दाखवते — कोणतेही side effects नाहीत, कोणतेही IO नाही. सर्व inputs parameters through येतात आणि output return होतो. हे testing सोपे करते आणि concurrency issues eliminate करते:

```
export class FeeCalculationService {
  calculate(
    feePlan: FeePlan,
    concessions: Concession[],
    lateFeePolicy: LateFeePolicy,
    asOfDate: Date,
  ): FeeBreakdown {
    // 1. Base fee from plan
    const baseFee = feePlan.totalAmount;

    // 2. Apply concessions (in priority order)
    const concessionAmount = concessions.reduce(
      (sum, c) => sum + c.apply(baseFee),
      0 as Money,
    );
  }
}
```

```

// 3. Calculate late fee if past due
const lateFee = lateFeePolicy.calculate(
  feePlan.dueDate,
  asOfDate,
  baseFee.minus(concessionAmount),
);

// 4. Tax (GST on applicable components)
const taxableAmount = baseFee.minus(concessionAmount);
const tax = TaxCalculator.gst(taxableAmount, feePlan.taxRate);

// 5. Final breakdown
return new FeeBreakdown({
  baseFee,
  concessionAmount,
  lateFee,
  tax,
  total: baseFee.minus(concessionAmount).plus(lateFee).plus(tax),
});
}
}

```

11. Repositories

Repositories हे aggregate persistence चे abstraction आहेत. Domain layer ला कसे data store होते हे माहित नसते — ते फक्त repository interface (port) शी interact करते. Concrete implementation (Prisma-backed) infrastructure layer मध्ये आहे. यामुळे domain layer pure राहते आणि testing मध्ये in-memory repository substitute केली जाऊ शकते.

11.1 Repository Interface Contract

प्रत्येक aggregate चा एक repository आहे. खालील interface methods standard आहेत — सर्व repositories यापैकी किमान methods implement करतात. Aggregate-specific queries (e.g., `findByRollNumber`) additional methods through add होतात:

Method	Returns	Notes
<code>findById(id)</code>	Student null	Single aggregate by id; returns null if not found or soft-deleted
<code>findByIdOrThrow(id)</code>	Student	Throws <code>NotFoundException</code> if missing — fail-fast path

Method	Returns	Notes
findBySpecification(spec, opts)	Student[]	Specification pattern query; opts = pagination/sort
findOneBySpecification(spec)	Student null	Convenience for unique matches
countBySpecification(spec)	number	For pagination totals; cheaper than full fetch
save(aggregate)	void	Insert or update (decides by presence of id); writes within UoW
delete(id)	void	Soft-delete (sets deletedAt); hard delete only via admin tooling
lock(id, mode)	Student	Pessimistic lock for concurrent writes (SELECT FOR UPDATE)
versionedSave(aggregate, expectedVersion)	void	Optimistic locking — fails on version mismatch
beginTransaction()	UoW	Returns unit-of-work handle for explicit control

11.2 Repository Rules

खालील rules repository implementation discipline सुनिश्चित करतात. यातील बऱ्याच rules ESLint ने enforce होतात — बाकी PR review मध्ये check होतात:

- Repository returns domain aggregate (not Prisma model)
- Repository does not publish events — that's the application service's job
- Repository methods are within UoW; commit happens at application layer
- One repository per aggregate root — not per entity
- Repository interface lives in domain/, implementation in infrastructure/
- Bulk operations use bulkSave() not looped save() — single SQL round-trip
- Read-only queries bypass aggregate — direct Prisma read for performance
- Caching layer wraps repository, transparent to caller
- No SQL in repository — use Prisma query builder; raw SQL only via approved DAO

- Repository never throws business exceptions — only infrastructure exceptions

11.3 Example — PrismaStudentRepository

Concrete implementation प्रत्येक interface method चे Prisma translation करते. Aggregate ↔ Prisma row mapping mapper class through होते जो three-way translation (DTO ↔ Domain ↔ Prisma) handle करतो:

```
@Injectable()
export class PrismaStudentRepository implements StudentRepository {
  constructor(private readonly prisma: PrismaClient) {}

  async findById(id: UUID, txn?: PrismaTransaction): Promise<Student | null>
  {
    const row = await this.prisma.student.findFirst({
      where: { id: id.toString(), deletedAt: null },
      include: { guardians: true, address: true },
    });
    return row ? StudentMapper.toDomain(row) : null;
  }

  async save(student: Student, txn?: PrismaTransaction): Promise<void> {
    const data = StudentMapper.toPersistence(student);
    const client = txn ?? this.prisma;

    if (student.isNew()) {
      await client.student.create({ data });
    } else {
      // Optimistic locking
      const result = await client.student.updateMany({
        where: { id: student.id.toString(), version: student.expectedVersion },
        data: { ...data, version: { increment: 1 } },
      });
      if (result.count === 0) {
        throw new ConflictException('VERSION_MISMATCH',
          'Student was modified by another transaction');
      }
    }
  }
}
```

12. CQRS

CQRS (Command Query Responsibility Segregation) हा pattern reads आणि writes ला separate करतो. PreOne मध्ये CQRS फक्त complex workflows मध्ये वापरला जाईल, प्रत्येक CRUD endpoint मध्ये नाही. हा ADR-012 चा decision आहे — simple CRUD operations साठी CQRS overhead justify होत

नाही, पण multi-aggregate workflows (admissions → enrollment → invoice) साठी command/query separation clarity देते.

12.1 When to Use CQRS

CQRS use करण्याचे clear criteria आहेत. हे criteria ADR-012 मध्ये documented आहेत आणि architect च्या approval नंतर वापरावे लागतात:

- Multi-aggregate workflows (3+ aggregates coordinated) — Admission approval involves Application + Student + Invoice.
- Read/write asymmetry — heavy reads with complex filters, simpler writes (e.g., student search).
- Independent scaling of reads — read replicas, denormalised projections (e.g., dashboard KPIs).
- Long-running business processes with state machine — application lifecycle, fee collection cycle.
- Event sourcing requirements — financial ledger, audit trail.

Simple CRUD (single aggregate, no workflow) साठी plain application service पुरेसा आहे — CreateStudentHandler सारखा command handler unnecessary complexity जोडतो.

12.2 Command Side

Commands state-changing operations represent करतात — Create, Update, Delete, Approve, Reject. प्रत्येक command immutable आहे, validated आहे, आणि त्याचा एकच handler आहे. Handler ची responsibility aggregate load करणे, method invoke करणे, repository through save करणे, आणि events publish करणे:

```

CreateStudentCommand (DTO)
  ↓
CommandBus.execute()
  ↓
CreateStudentHandler
  ↓ (loads + invokes + saves + publishes)
StudentAggregate.create() // pure domain logic
  ↓
StudentRepository.save() // Prisma persistence
  ↓
Outbox → StudentCreatedEvent // async event publishing

```

Command Catalog

Command	Module	Purpose	Event Published
CreateStudentComm and	Identity	Create new student record + initial profile	StudentCreatedEvent
EnrollStudentComma nd	Academics	Enroll student in section for academic year	StudentEnrolledEven t, InvoiceGeneratedEve nt
MarkAttendanceCom mand	Attendance	Mark daily attendance for student	AttendanceMarkedE vent
GenerateInvoiceCom mand	Finance	Generate fee invoice for student	InvoiceGeneratedEve nt
ReceivePaymentCom mand	Finance	Record payment against invoice	PaymentReceivedEve nt, ReceiptIssuedEvent
ApproveAdmissionC ommand	Admissions	Approve pending admission application	AdmissionApprovedE vent, StudentCreatedEvent
PublishReportCardCo mmand	Academics	Publish term report card to parents	ReportCardPublished Event
ScheduleInterviewCo mmand	HR	Schedule interview with candidate	InterviewScheduledE vent
RaisePurchaseReque stCommand	Inventory	Create PR for procurement	PurchaseRequestRais edEvent
IssueGoodsComman d	Inventory	Issue goods from store to department	GoodsIssuedEvent, StockReducedEvent

12.3 Query Side

Queries read-only operations represent करतात — Get, Search, List, Count. Queries aggregate layer ला touch करत नाहीत — ते direct Prisma read model वापरतात performance साठी. यामुळे read path फक्त DB + cache + DTO mapping इतका simple आहे:

```
GetStudentQuery
↓
QueryBus.execute()
```

```

↓
GetStudentHandler
  ↓ (read-optimized path)
  PrismaStudentReadModel.find()
    ↓
    StudentListItemDto // no aggregate hydration

```

Query Catalog

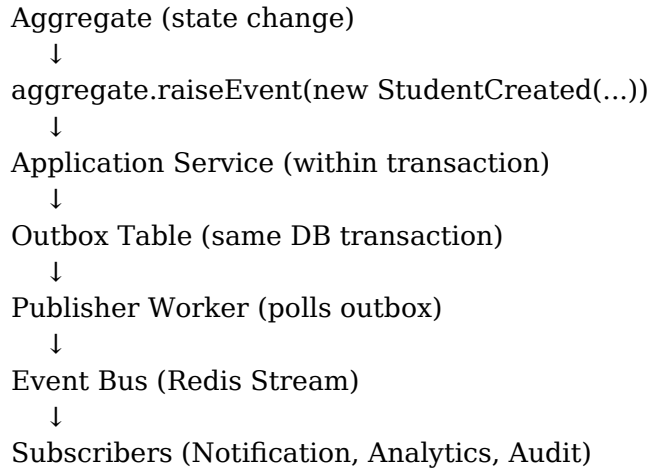
Query	Module	Purpose	Returns
GetStudentByIdQuery	Identity	Fetch full student profile	StudentResponseDto
SearchStudentsQuery	Identity	Full-text + filter search	Paginated<StudentListItemDto>
GetAttendanceReportQuery	Attendance	Date-range attendance summary	AttendanceReportDto
GetFeeOutstandingQuery	Finance	Outstanding fees per student/class	FeeOutstandingDto[]
GetDashboardKpiQuery	Reports	Dashboard KPIs by role/branch	DashboardKpiDto
GetStaffOnLeaveQuery	HR	Staff on leave today/this week	StaffLeaveDto[]
GetLowStockItemsQuery	Inventory	Items below reorder level	LowStockDto[]
GetAdmissionPipelineQuery	Admissions	Admission funnel metrics	AdmissionPipelineDto
GetLedgerQuery	Finance	GL ledger entries by period	Paginated<LedgerEntryDto>
GetAuditTrailQuery	Platform	Audit trail by entity/user/date	Paginated<AuditLogDto>

13. Domain Events

Domain Events हे 'something happened in the domain' चा formal way आहे. Aggregate जेव्हा state change करतो तेव्हा event raise करतो, आणि तो event bus वर publish होतो. Subscribers (within same module किंवा cross-module via integration events) त्या event ला react करतात. यामुळे aggregates एकमेकांशी loosely coupled राहतात.

13.1 Event Flow

Event चा flow खालीलप्रमाणे आहे. Outbox pattern reliability सुनिश्चित करतो — event same transaction मध्ये DB ला लिहला जातो, नंतर background publisher त्याला Redis Stream वर push करतो. यामुळे events कधीच lost होत नाहीत:



13.2 Event Catalog

खालील table PreOne च्या सर्व domain events ची authoritative list आहे. नवीन event introduce करताना या table मध्ये entry जोडावी लागेल आणि event schema versioned असणे mandatory आहे:

Event	Module	Trigger	Payload	Subscribers
StudentCreated	identity	New student created	Student id, name, branch, section, createdBy	Sync: parent welcome email; Async: CRM lead conversion
StudentUpdated	identity	Student profile updated	Diff of changed fields, updatedBy	Sync: cache invalidation; Async: audit log
StudentPromoted	academics	Student promoted to next class	From class, to class, academic year	Async: new fee plan, new section assignment
StudentTransferred	identity	Student transferred to another branch	From branch, to branch, reason	Async: archival at old branch, new record at new branch

Event	Module	Trigger	Payload	Subscribers
AdmissionApplicationSubmitted	admissions	New admission enquiry	Application id, student name, parent contact	Sync: confirmation SMS; Async: counsellor assignment
AdmissionApproved	admissions	Admission application approved	Application id, student id, branch	Async: StudentCreated, InvoiceGenerated, welcome email
AdmissionRejected	admissions	Admission application rejected	Application id, reason	Async: rejection email, CRM status update
AttendanceMarked	attendance	Daily attendance marked	Student, date, status, markedBy	Sync: parent SMS; Async: monthly summary
InvoiceGenerated	finance	Fee invoice raised	Invoice id, student, amount, due date	Async: email + WhatsApp, reminder schedule
PaymentReceived	finance	Payment recorded against invoice	Invoice id, amount, method, receiptNo	Sync: receipt SMS; Async: ledger entry, GST return prep
PaymentFailed	finance	Payment attempt failed	Invoice id, amount, failure reason	Async: failure notification, retry schedule
ReportCardPublished	academics	Term report card published	Student, term, sections, grades	Async: parent email + push notification
ObservationApproved	academics	Teacher observation approved	Observation id, student, milestone	Async: parent notification, milestone tracking

Event	Module	Trigger	Payload	Subscribers
StaffOnboarded	hr	New staff onboarded	Staff id, name, role, branch	Sync: credentials setup; Async: asset assignment
StaffOffboarded	hr	Staff offboarded	Staff id, exitDate, reason	Async: asset recovery, access revoke, final settlement
LeaveApplied	hr	Leave request submitted	Staff id, leave type, dates	Sync: manager notification; Async: calendar block
LeaveApproved	hr	Leave request approved	Staff id, leave type, dates, approver	Async: substitute assignment
PurchaseRequestRaised	inventory	PR created	PR id, items, department	Async: manager approval workflow
GoodsIssued	inventory	Goods issued from store	Items, qty, to department	Async: stock recalc, ledger impact
StockLow	inventory	Item below reorder level	Item, current qty, reorder level	Async: PR auto-creation if rule exists
Announcement Published	communication	Announcement broadcast	Audience, channels, content	Async: SMS/WhatsApp /Email/Push fan-out
MessageSent	communication	Chat message sent	Room, sender, message	Sync: WebSocket push; Async: push for offline users
BackupCompleted	platform	Daily backup finished	Database size, duration, location	Async: integrity check, alert on failure

Event	Module	Trigger	Payload	Subscribers
FeatureFlagToggled	platform	Feature flag changed	Flag key, old value, new value, changedBy	Sync: cache bust; Async: audit log

13.3 Event Schema Rules

- Event names: past-tense verb (StudentCreated, not CreateStudent).
- Events are immutable — once published, never modified.
- Event payload includes: eventId (UUID v7), occurredAt, tenantId, userId, aggregateId, version.
- Schema versioned via .v1, .v2 suffix — backward-compatible evolution only.
- Subscribers must be idempotent — same event processed twice must yield same result.
- Events published after transaction commit — never before.
- Event payload size max 64 KB — larger payloads use reference (URL) instead of inline data.
- PII in events encrypted at rest (Redis AES); access logged.

14. Integration Events

Integration Events हे cross-bounded-context communication चा mechanism आहे. Domain events (Section 13) single bounded context अंतर्गत असतात, integration events त्यांच्या translation होतात जेव्हा दुसरा context त्याचा subscriber असतो. उदा. StudentCreated event Identity module मध्ये raise होतो, पण Finance module ला new student ची जाणीव हवी असेल तर Identity एक StudentEnrolled integration event publish करतो जो Finance subscribe करतो.

14.1 Integration Event Catalog

Event	Producer → Consumer	Purpose	Delivery
StudentEnrolled.v1	Identity → Finance	Trigger fee plan creation	Sync (HTTP) — finance needs immediate response

Event	Producer → Consumer	Purpose	Delivery
InvoiceGenerated.v1	Finance → Identity	Notify student record of pending invoice	Async (Redis Stream) — eventual consistency
PaymentReceived.v1	Finance → Communication	Trigger receipt notification	Async (Redis Stream)
AdmissionApproved.v1	Admissions → Identity	Create student record	Sync (HTTP) — admissions waits for student id
AttendanceMarked.v1	Attendance → Communication	Trigger parent SMS	Async (Redis Stream)
StaffOnboarded.v1	HR → Identity	Create user account	Sync (HTTP) — HR waits for credentials
StockLow.v1	Inventory → Procurement	Auto-create PR if rule active	Async (Redis Stream)
ReportCardPublished.v1	Academics → Communication	Notify parents via email + push	Async (Redis Stream)
AnnouncementPublished.v1	Communication → Platform	Record in audit log	Async (Redis Stream)
FeatureFlagChanged.v1	Platform → All	Cache bust notification	Pub/Sub (Redis) — all subscribers invalidate

14.2 Integration Patterns

Integration events च्या two delivery patterns आहेत — sync (HTTP) आणि async (Redis Stream). Pattern choice ADR-015 मध्ये per-integration basis decide केला जातो. मुख्य criteria: काय consumer चा immediate response पाहिजे आहे?

- **Sync (HTTP):** Producer HTTP call करतो consumer च्या internal endpoint वर, response waits. Used when producer ला consumer चा result पाहिजे (e.g., Admissions → Identity for student ID).
- **Async (Redis Stream):** Producer event Redis Stream वर publish करतो, consumer खाली subscribe करतो. Used when eventual consistency acceptable (e.g., PaymentReceived → Communication).

- **Saga Pattern:** Long-running multi-step transactions ज्यात प्रत्येक step own transaction आहे. Compensating actions rollback साठी. Used for admission → enrollment → invoice flow.
- **Outbox Pattern:** Producer event DB outbox table मध्ये same transaction मध्ये writes करतो. Background publisher outbox ला drain करून Redis Stream वर push करतो. Reliability सुनिश्चित करतो.

14.3 Integration Rules

- Integration events are versioned (.v1, .v2) — backward-compatible schema evolution
- Sync integration = HTTP call within same request — must have timeout + circuit breaker
- Async integration = Redis Stream — at-least-once delivery; consumer must be idempotent
- Producer publishes to outbox table; publisher worker drains to Redis Stream
- Consumer maintains last-read position in Redis (consumer group offset)
- Retry policy: 3 retries with exponential backoff (1s, 5s, 25s); then DLQ
- Poison messages (failed 3x) routed to DLQ for manual inspection
- Schema changes require new version — never modify existing event schema in-place
- Event payload includes schema URL for runtime validation by consumer
- All events include: eventId (UUID), occurredAt (ISO), tenantId, branchId, userId

15. Background Jobs

Background Jobs हे long-running किंवा deferred work handle करतात. Request lifecycle मध्ये synchronous होऊ नये असे work BullMQ च्या through offload केले जाते — उदा. PDF report generation (30s+), bulk WhatsApp send (rate-limited), nightly backups. यामुळे API responses fast राहतात आणि user कोणताही wait करत नाही.

15.1 BullMQ Queue Catalog

PreOne मध्ये 12 dedicated queues आहेत, प्रत्येक queue ची priority आणि rate limit वेगवेगळी आहे. हे separation critical work (notifications) कमी priority work (reports) पासून isolate करते, ज्यामुळे एक queue overload झाल्यास दुसरी affected होत नाही:

Queue	Purpose	Priority	Rate Limit	Notes
notification	Parent SMS / push notifications	High	1000/min	Burst during attendance mark, fee due, report publish
email	Transactional + marketing emails	Medium	500/min	Bulk admissions email uses separate child queue
whatsapp	WhatsApp Business API messages	Medium	100/min	WhatsApp rate limits are stricter than SMS
sms	Transactional SMS via DLT-approved senders	High	2000/min	DLT template validation before send
report	PDF/Excel report generation	Low	50/min	Long-running (30s-5min) — sandboxed
ai	AI/LLM inference jobs (chatbot, summaries)	Medium	100/min	Token-bucket per tenant to control cost
image-processing	Image resize, watermark, thumbnail	Medium	200/min	Sharp library; CPU-bound
backup	Daily DB + S3 backup	Low	1/day (off-peak)	03:00 IST; full backup + incremental
audit-cleanup	Audit log archival to S3 Glacier	Low	1/week	Move logs older than 1 year
invoice-reminder	Fee due reminders	Medium	1000/hr burst	Cron at 09:00 IST daily

Queue	Purpose	Priority	Rate Limit	Notes
report-card-publish	Bulk report card publishing	Medium	500/hr	Term-end spike; pre-generated PDFs
data-export	User-requested data exports	Low	50/min	Async upload to S3 + email link

15.2 Retry Policy

Background jobs fail होऊ शकतात — external service down, DB timeout, network glitch. Retry policy यांना gracefully handle करतो. खालील table policy चे detail देते. Exponential backoff ने retry ची frequency कमी होते ज्यामुळे downstream service ला recover होण्याची मुदत मिळते:

Aspect	Value	Notes
Max Attempts	3 retries after initial attempt	Total 4 attempts before DLQ
Backoff Strategy	Exponential — delay = $2^{\text{attempt}} \times 1\text{s}$, capped at 60s	1s, 2s, 4s, 8s, 16s, 32s, 60s, ...
Jitter	$\pm 20\%$ random jitter on backoff	Avoids thundering herd on retry
Dead Letter Queue	{queueName}:dead	Manual inspection; alert via PagerDuty
Failure Alert	Slack webhook after 3 consecutive failures in same queue	Auto-page on-call engineer for critical queues (notification, sms, backup)
Idempotency	Job ID = hash(payload + tenantId)	Duplicate job detection; safe retry
Sandboxed Processors	BullMQ sandbox mode for report/ai queues	Crash isolation; parent process survives
Cron Jobs	Defined via @nestjs/schedule + BullMQ repeatable	Single-instance enforced via Redis lock

15.3 Job Lifecycle

प्रत्येक job खालील lifecycle stages through जाते. हे visibility debugging आणि monitoring साठी critical आहे — Bull Board UI सर्व active/waiting/delayed/failed jobs दाखवते:

```

Producer (e.g., Controller or Handler)
  ↓ queue.add('job-name', payload, opts)
Waiting (in Redis sorted set)
  ↓ worker picks up
Active (being processed)
  ↓ on success
Completed (retained 24h for inspection)
  -- OR --
  ↓ on failure
Delayed (backoff timer running)
  ↓ retries up to maxAttempts
Failed (moved to DLQ after maxAttempts)
  ↓ alert triggered
Manual inspection via Bull Board

```

16. Cache Strategy

Redis हा PreOne चा primary cache layer आहे — sub-ms reads, persistence, pub/sub, आणि BullMQ backend सर्व एकाच instance वर. Cache strategy ADR-018 मध्ये defined आहे आणि त्याचा core principle म्हणजे 'explicit invalidation over TTL' — predictable behaviour ची preference convenience वर. Permission cache आणि version-based invalidation ADR मध्ये define केलेले आहेत.

16.1 Cache Layers

PreOne मध्ये 10 distinct cache layers आहेत, प्रत्येक layer ची specific TTL आणि invalidation strategy आहे. खालील table सर्व layers ची catalog करते:

Layer	Key Pattern	TTL	Invalidation	Notes
Permission Cache	user_id → [permissions]	300s TTL	Invalidated on role change, permission grant/revoke	Hot path on every request
Menu Cache	role_id → menu tree	1h TTL	Invalidated on menu config change	Avoids DB hit on every page load
User Session	session_id → user context	24h TTL sliding	Deleted on logout	Stored as Redis hash
Dashboard KPIs	tenant+branch+role → KPI bundle	60s TTL	Auto-expire; manual bust via admin tool	Refreshed by background job

Layer	Key Pattern	TTL	Invalidation	Notes
Settings	tenant+branch → settings object	1h TTL	Versioned by settings_version counter	Bump version on update
Feature Flags	tenant → flag map	300s TTL	Invalidated on flag change event	Pub/Sub broadcast to all instances
DTO Validation Schema	schema name → compiled Zod schema	Process lifetime	Recompile on deploy	In-process Map; not Redis
Compiled Casbin Model	tenant → enforcer	1h TTL	Invalidated on policy change	Hot path on auth check
Pre-signed S3 URLs	object key → URL	55 min TTL (URL valid 1h)	Auto- regenerate	Avoids S3 API calls
Rate Limit Counters	ip+route → counter	60s window	Auto-expire	Sliding window via Redis sorted set

16.2 Cache Patterns

वेगवेगळ्या use cases साठी वेगवेगळे patterns वापरले जातात. Pattern choice data consistency requirements + read/write ratio वर depend करते:

Pattern	Flow	Use Case
Read-Through	App → cache → DB → cache → app	Hot reads; transparent to caller
Write-Through	App → cache + DB (sync) → app	Strong consistency; slightly higher write latency
Write-Behind	App → cache → ack; async DB write	High write throughput; risk of data loss on crash
Cache-Aside	App checks cache; on miss loads from DB	Most common; explicit cache management
Refresh-Ahead	Background job refreshes before expiry	Prevents cache stampede on hot keys

16.3 Cache Invalidation Rules

Cache invalidation हे computer science च्या two hard problems पैकी एक आहे. खालील rules predictable invalidation सुनिश्चित करतात:

- Explicit invalidation preferred over TTL — predictability over convenience
- Versioned cache keys (e.g., user_perms:{userId}:v{version}) allow atomic invalidation
- Pub/Sub broadcast for multi-instance invalidation
- Cache stampede protection via single-flight pattern (only one DB fetch per key)
- Never cache PII without encryption at rest (Redis AES enabled)
- Cache hit ratio target: $\geq 90\%$ for permission/menu; $\geq 70\%$ for KPIs
- Monitoring: per-cache hit/miss/error metrics emitted to Prometheus

16.4 Permission Cache Deep-Dive

Permission cache हा सर्वात hot path आहे — प्रत्येक request वर permission check होतो, ज्यामुळे cache hit ratio $\geq 90\%$ पाहिजे. Permission cache ची architecture खालीलप्रमाणे आहे. Version-based invalidation atomic cache busting सुनिश्चित करते — role change झाल्यावर user_perms:{userId}:v{version} चा version bump होतो आणि जुनी entry TTL expire होऊन नवीन entry populate होते:

```

Request arrives (userId in JWT)
↓
Cache key: user_perms:{userId}:v{version}
↓
Redis GET
↓ HIT
Return permissions
-- OR --
↓ MISS
Casbin enforcer load (DB query)
↓
Redis SET with 300s TTL
↓
Return permissions

```

// Invalidation on role change:

```

UPDATE users SET perms_version = perms_version + 1 WHERE id = ?
// Old cache key naturally expires; new key populated on next request

```

17. Transactions

Transactions हे data consistency चा foundation आहेत. PreOne चा core principle: 'Aggregate = Transaction Boundary'. एका aggregate चा save हा एक transaction आहे — multiple aggregates एकाच transaction मध्ये save होणे anti-pattern आहे. जर multi-aggregate operation हवा असेल तर Saga pattern वापरा — प्रत्येक step own transaction, rollback via compensating action.

17.1 Transaction Patterns

वेगवेगळ्या scenarios साठी वेगवेगळे transaction patterns वापरले जातात. खालील table सर्व patterns ची catalog करते:

Pattern	Mechanism	Use Case	Flow
Single-Aggregate Transaction	ACID via Prisma \$transaction	Default — 95% of writes	Begin → save aggregate → commit
Multi-Aggregate Saga	Orchestration via state machine + compensating actions	Cross-bounded-context flows	Each step = own transaction; rollback via compensation
Outbox Pattern	DB transaction writes events to outbox table; publisher drains to stream	Reliable event publishing	Same transaction as aggregate write
Optimistic Locking	version column checked on save	Low-contention aggregates	WHERE version = expected; throw on 0 rows affected
Pessimistic Locking	SELECT ... FOR UPDATE	High-contention (concurrent fee edits)	Hold lock for shortest possible duration
Idempotency	idempotency_key table; unique constraint	All POST/PUT	Reject duplicate; return cached response
Read-Only Transaction	Prisma read without write	All GET	No lock acquired; replica routing possible

17.2 Transaction Rules

- Aggregate = Transaction Boundary — never split one aggregate across transactions
- No cross-aggregate transactions — use saga or domain events instead
- Always set transaction timeout (default 5s; longer for batch)
- Never call external services (HTTP, S3, email) within transaction — do after commit
- Rollback on any exception — let it propagate to global filter
- Transaction isolation level: READ COMMITTED (default); SERIALIZABLE only for critical billing
- Max transaction duration: 5s; longer = suspect deadlock — alert
- Test transactions under load to detect deadlocks (Testcontainers + concurrent Vitest)
- Log transaction begin/commit/rollback with traceId for observability

17.3 Example — Admission Approval Saga

Admission approval हे multi-aggregate workflow आहे — Application, Student, FeePlan, Invoice या चार aggregates involve होतात. एकाच transaction मध्ये सर्व save करणे anti-pattern असेल. खालील saga pattern चे implementation दाखवते — प्रत्येक step own transaction, failure वर compensating action:

```

Step 1: Approve Application (admissions module)
Txn 1: application.status = APPROVED; application.save()
Event: AdmissionApproved
↓
Step 2: Create Student (identity module)
Txn 2: student = StudentFactory.fromApplication(application); student.save()
Event: StudentCreated
↓ (compensation on failure: mark application for manual review)
↓
Step 3: Create FeePlan (finance module)
Txn 3: feePlan = FeePlanFactory.forStudent(student, branch); feePlan.save()
Event: FeePlanCreated
↓ (compensation: soft-delete student, alert ops)
↓
Step 4: Generate First Invoice (finance module)
Txn 4: invoice = student.generateInvoice(feePlan, term1); invoice.save()
Event: InvoiceGenerated
↓ (compensation: delete feePlan, alert ops)
↓
Step 5: Send Welcome Email (communication module, async)
Job: enqueue welcome email + WhatsApp message

```

```
// If any step fails after max retries:
// - Saga state = FAILED
// - Compensating actions execute in reverse order
// - PagerDuty alert to on-call engineer
```

18. Validation

Validation हे defense-in-depth strategy आहे. PreOne मध्ये 6 validation layers आहेत — प्रत्येक layer वेगवेगळ्या type ची invalid input catch करते. यामुळे invalid data कधीच database पर्यंत पोहोचत नाही. Layer-by-layer validation fail-fast principle follow करते — जितक्या लवकर error catch होईल तितका clear error message.

18.1 Validation Layers

Layer	Tool	Where	Error Code	What It Catches
DTO Validation	class-validator + Zod	Request pipe	422 Unprocessable Entity	Schema shape, required fields, type coercion
Domain Validation	Value object constructors	Domain layer	DomainException	Business invariants (e.g., valid phone, age range)
Business Rule Validation	Specifications + Policies	Application layer	BusinessException	Cross-entity rules (e.g., unique roll number in section)
Database Constraints	NOT NULL, UNIQUE, FK, CHECK	Database layer	ConflictException	Last line of defense — should never trigger if above layers work
Authorization Validation	Guards + decorators	Request pipe (after DTO)	AuthorizationException	Permission + data scope check
Rate Limit Validation	Throttler guard	Request pipe (first)	429 Too Many Requests	Per-IP + per-user counters

18.2 Validation Flow

Request चा flow खालील stages through जातो. प्रत्येक stage वेगवेगळ्या type ची validation perform करते. जर early stage मध्ये fail झाले तर later stages skip होतात — यामुळे wasted work कमी होते:

1. Request arrives at controller
2. Throttler guard checks rate limit (429 if exceeded)
3. JwtAuthGuard authenticates (401 if invalid token)
4. PermissionsGuard checks authorization (403 if denied)
5. ValidationPipe runs DTO class-validator (422 if invalid)
6. Zod validation pipe runs schema (422 if invalid)
7. Idempotency check (409 if duplicate)
8. Handler invoked
9. Domain validation throws DomainException (400)
10. Business rule check throws BusinessException (409)
11. Repository save - DB constraint violation throws ConflictException (409)
12. Success - ResponseDto returned (200/201)

Each layer has a specific responsibility and a specific error code on failure. This separation makes debugging easier — the error code tells you which layer failed, and the field-level details array pinpoints the exact field.

18.3 Validation Best Practices

- Validate at the boundary — never trust external input.
- Fail fast — cheapest check first (rate limit before DB query).
- Return specific error codes — not generic 'INVALID_INPUT'.
- Field-level errors for ValidationException — array of { field, code, message }.
- Never swallow exceptions — let them propagate to global filter.
- Log validation failures at warn level — useful for UX improvement.
- Domain layer validates via value object constructors — fail on construction.
- Business rules in specifications — composable, testable.

19. Exception Handling

Exception handling हे user experience चा critical part आहे. Generic error messages ("Something went wrong") user कडून frustration create करतात. PreOne चा standardised error response shape specific error codes, localised messages, आणि traceId correlation देतो — ज्यामुळे user ला काय चुकले आणि support ला कसे debug करायचे हे clear होते.

19.1 Exception Types

Exception	HTTP Status	When	Example Codes	Recovery
ValidationException	422	DTO schema violation	FIELD_REQUIRED, INVALID_EMAIL, OUT_OF_RANGE	Client error — retry with corrected input
BusinessException	409	Business rule violation	ROLL_NUMBER_TAKEN, INVOICE_ALREADY_PAID, ADMISSION_CLOSED	Client must change request — retry with different data
AuthorizationException	403	Insufficient permissions	PERMISSION_DENIED, SCOPE_VIOLATION	Client must request elevated access
AuthenticationException	401	Invalid or expired token	TOKEN_EXPIRED, TOKEN_INVALID, SESSION_REVOKED	Client must re-authenticate
NotFoundException	404	Resource does not exist	STUDENT_NOT_FOUND, INVOICE_NOT_FOUND	Client must verify ID
ConflictException	409	Concurrent modification	VERSION_MISMATCH, DUPLICATE_KEY	Client may retry with fresh fetch
RateLimitException	429	Rate limit exceeded	RATE_LIMIT_EXCEEDED	Client must back off (Retry-After header)
ExternalServiceException	502	Downstream service failure	S3_UNAVAILABLE, SMS_GATEWAY_DOWN	Server retries via BullMQ

Exception	HTTP Status	When	Example Codes	Recovery
InfrastructureException	500	DB/Redis failure	DB_CONNECTION_LOST, REDIS_TIMEOUT	Server alert — on-call paged
DomainException	400	Invalid value object construction	INVALID_PHONE, INVALID_MONEY	Client must provide valid value

19.2 Standard Error Response

PreOne च्या सर्व error responses खालील shape follow करतात. Global ExceptionFilter हे shape enforce करतो — कोणताही unhandled exception या shape मध्ये wrap होतो. traceId logs शी correlate करतो ज्यामुळे support tickets मध्ये user फक्त traceId quote करतो आणि engineer full context पुन्हा reconstruct करू शकतो:

```
{
  "success": false,
  "errorCode": "STUDENT_NOT_FOUND",
  "message": "Student with ID abc-123 does not exist",
  "traceId": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "details": [],
  "timestamp": "2026-07-14T10:30:45.123Z",
  "path": "/v1/students/abc-123"
}
```

Response Field Reference

Field	Type	Purpose
success	boolean	Always false for errors
errorCode	string	Stable machine code — never localize
message	string	Human-readable, localized per Accept-Language
traceId	UUID	Correlates to logs — quote in support tickets
details	object[]	Field-level errors (for ValidationException only) — { field, code, message }

Field	Type	Purpose
timestamp	ISO-8601	When the error occurred
path	string	Request path that triggered the error

19.3 Global Exception Filter

Global ExceptionFilter सर्व exceptions catch करतो आणि त्यांना standard response shape मध्ये translate करतो. खालील snippet filter चा essence दाखवते — full implementation `app/filters/all-exceptions.filter.ts` मध्ये आहे:

```
@Catch()
export class AllExceptionsFilter implements ExceptionFilter {
  catch(exception: unknown, host: ArgumentsHost) {
    const ctx = host.switchToHttp();
    const response = ctx.getResponse();
    const request = ctx.getRequest();
    const traceId = request.traceId;

    let status: number;
    let body: ErrorResponse;

    if (exception instanceof DomainException) {
      status = 400;
      body = this.buildBody(exception.code, exception.message, traceId,
request.path);
    } else if (exception instanceof BusinessException) {
      status = 409;
      body = this.buildBody(exception.code, exception.message, traceId,
request.path);
    } else if (exception instanceof HttpException) {
      status = exception.getStatus();
      body = this.buildBody(exception.name, exception.message, traceId,
request.path);
    } else {
      // Unhandled - log + 500
      logger.error({ exception, traceId });
      status = 500;
      body = this.buildBody('INTERNAL_ERROR', 'An unexpected error
occurred', traceId, request.path);
    }

    response.status(status).json(body);
  }
}
```

20. Security

Security हे PreOne सारख्या enterprise SaaS साठी non-negotiable आहे. DPDP Act 2023, ISO 27001, आणि client contract SLAs strict security requirements impose करतात. PreOne चा security approach defense-in-depth आहे — multiple layers मध्ये controls, ज्यामुळे एक layer fail झाला तरी दुसरा defense देतो. हे section ADR-022 मध्ये documented controls चा summary आहे.

20.1 Security Controls Catalog

Control	Implementation	Purpose
JWT Authentication	Access token 15 min (RS256 signed); refresh token 30 d (rotated, HttpOnly cookie)	Stateless; horizontal scaling friendly
Refresh Token Rotation	Each refresh issues new token; old one blacklisted	Detects token theft — both tokens used = revoke all
RBAC	Casbin policies; role → permissions matrix	Fine-grained; per-route enforcement via @Permissions decorator
Data Scope Validation	Every query scoped by tenantId + branchId from JWT	Tenant isolation at ORM layer (Prisma middleware)
Tenant Isolation	PostgreSQL Row-Level Security (RLS) policies	Defense in depth — even buggy query returns only tenant data
Branch Isolation	Branch ID in JWT; query filter enforced	Cross-branch access requires explicit permission
Academic Year Isolation	Academic year ID in context; filter on every academic query	Prevents mixing current + archived data
Rate Limiting	Per-IP (anonymity) + per-user (authenticated) limits	Default 100 req/min; stricter on auth routes (5/min)
Input Sanitisation	DOMPurify on rich text; parameterised queries (Prisma)	XSS + SQL injection prevention
PII Protection	PII fields encrypted at rest (pgcrypto); masked in logs	DPDP Act compliance

Control	Implementation	Purpose
Audit Logging	Every state-changing action logged with user + diff	Compliance + forensic analysis
Secrets Management	AWS Secrets Manager; rotated quarterly	No secrets in env files or code
CSP / Helmet	Content-Security-Policy + OWASP headers	XSS mitigation in browser
CORS	Allowlist of origins per environment	Production: only preone.app + subdomains
HSTS	Force HTTPS; 1-year max-age; includeSubDomains	Prevents protocol downgrade
Dependency Scanning	Snyk + npm audit in CI; fail on high+ vulns	Supply chain security
SAST	SonarCloud on every PR	Static analysis — bugs + vulnerabilities
DAST	OWASP ZAP scan on staging weekly	Runtime vulnerabilities

20.2 Authentication Flow

JWT authentication चा flow खालीलप्रमाणे आहे. Access tokens short-lived (15 min) आहेत — compromise झाल्यास limited damage. Refresh tokens long-lived (30 d) पण rotated — प्रत्येक refresh ने new token issue होतो आणि old one blacklist होतो. दोन्ही tokens वापरले गेल्यास (concurrent use) theft detect होते आणि सर्व sessions revoke होतात:

```

Login Request (email + password)
↓
Verify password (argon2 hash compare)
↓ valid
Issue Access Token (15 min, RS256 signed)
Issue Refresh Token (30 d, rotated, HttpOnly cookie)
↓
Client uses Access Token for API calls
↓ on expiry
Client calls /auth/refresh with Refresh Token
↓
Server verifies + rotates: new Access + new Refresh
↓ old refresh token blacklisted
Continue session
-- OR --
↓ on logout

```

Both tokens blacklisted



Redis session record deleted

20.3 PII Protection

PII (Personally Identifiable Information) fields database मध्ये encrypted at rest (pgcrypto) store होतात. Logs मध्ये mask होतात. Audit trail मध्ये access logged होतो. DPDP Act Section 17 नुसार data principal (user) ला access, correction, आणि erasure चा right आहे — हे rights PreOne च्या DSAR (Data Subject Access Request) workflow through honored होतात.

21. Multi-Tenant Strategy

PreOne हा multi-tenant SaaS आहे — एकच instance 10,000+ schools serve करतो. Tenant isolation हे security चा foundation आहे — एक school च data दुसऱ्या school ला कधीच visible होऊ नये. PreOne चा approach 'shared schema with discriminator' आहे — कोणत्याही table मध्ये tenant_id column आहे आणि प्रत्येक query मध्ये WHERE tenant_id = ? filter mandatory आहे.

21.1 Isolation Layers

Tenant isolation 5 layers मध्ये enforce होते — defense in depth. एक layer fail झाला तरी दुसरा defense देतो. हे layers ADR-025 मध्ये documented आहेत:

Layer	Mechanism	Notes
Tenant Discriminator	tenant_id column on every table	Index on (tenant_id, ...); partitioning-ready
JWT Claim	tenantId in JWT payload	Populated on login; verified on every request
Prisma Middleware	Auto-inject tenantId WHERE clause	Catches missing filters; logs violation
PostgreSQL RLS	Row-Level Security policy per tenant	Last line of defense; failsafe if Prisma bypassed
Tenant Config	Per-tenant settings (logo, colors, modules enabled)	Cached in Redis; hot reload on change
Tenant Onboarding	New tenant = new schema? No — shared schema with discriminator	Cost-efficient; migration simplicity

Layer	Mechanism	Notes
Cross-Tenant Admin	Platform admin role bypasses RLS (BYPASSRLS)	Only platform team has this — audited
Tenant Backup/Restore	Per-tenant logical backup (pg_dump with WHERE)	Customer data portability
Tenant Deletion	Soft delete + 90-day retention + then hard purge	DPDP Act right to erasure compliance

21.2 Isolation Scope

PreOne मध्ये tenant हा primary isolation scope आहे, पण त्यावर दोन additional scopes आहेत — Branch आणि Academic Year. Branch isolation optional आहे (multi-branch schools साठी), Academic Year isolation academic queries साठी mandatory आहे:

Scope	Mandatory?	Applies To	Source
Tenant	Mandatory	Every query, every table	tenantId from JWT
Branch	Optional (multi-branch schools)	Branch-specific tables	branchId from JWT or query param
Academic Year	Optional (academic-year-aware queries)	Academic tables	academicYearId from context

21.3 PostgreSQL RLS Policy

Row-Level Security (RLS) हा last line of defense आहे — जर Prisma middleware bug असेल किंवा developer raw query लिहिली तरी RLS policy ensure करते की cross-tenant data return होत नाही. खालील snippet typical RLS policy दाखवते. tenant_id session variable through set होते जेव्हा connection acquire होते:

```
-- Enable RLS on students table
ALTER TABLE students ENABLE ROW LEVEL SECURITY;

-- Policy: users can only see rows from their tenant
CREATE POLICY tenant_isolation ON students
FOR ALL
USING (tenant_id = current_setting('app.tenant_id')::uuid);

-- Platform admin bypasses RLS (audited)
CREATE ROLE platform_admin BYPASSRLS;
```

```
-- Application sets tenant_id on connection acquire:
-- SET app.tenant_id = 'abc-123-...';
-- SET app.user_id = 'user-456-...';
-- SET app.branch_id = 'branch-789-...';
```

22. Logging

Logging हे observability चा primary pillar आहे. PreOne मध्ये structured JSON logging (Pino) वापरली जाते — free-text logs कठीण parse करायला, त्याऐवजी structured fields enable करतात efficient querying, alerting, आणि dashboarding. प्रत्येक log entry traceId सह correlate होते ज्यामुळे single request चा complete journey एका query ने fetch केला जाऊ शकतो.

22.1 Log Fields

प्रत्येक log entry खालील fields contain करते. हे fields structured format मध्ये emit होतात ज्यामुळे Loki / Elasticsearch मध्ये indexed queryable होतात. PII fields redact plugin ने automatically mask होतात — developer ला manually redact करण्याची गरज नाही:

Field	Type	Purpose
timestamp	ISO-8601 with ms	When the log was generated
level	info warn error debug trace	Severity
traceId	UUID	Distributed trace correlation
spanId	UUID	Sub-span within trace
userId	UUID	Authenticated user (or null for anon)
tenantId	UUID	Tenant context
branchId	UUID	Branch context (optional)
module	string	Module name (identity, finance, etc.)
method	string	Method/class generating the log
message	string	Human-readable message
payload	object	Structured data (never PII)
error	object	Stack trace + code (only on error level)

Field	Type	Purpose
duration	number	Operation duration in ms (for performance logs)
requestId	UUID	Per-request correlation (set by middleware)

22.2 Log Levels

Log levels severity indicate करतात. Production मध्ये trace आणि debug disabled असतात — ते staging / dev मध्ये enable होतात. warn आणि error ला alerting rules attach आहेत — high volume = automated alert:

Level	When	Production Default
trace	Very detailed — SQL queries, cache hits/misses	Disabled in production
debug	Detailed diagnostic info	Enabled on staging; per-route enable on prod via feature flag
info	Normal operation — request received, command executed, event published	Always enabled
warn	Unexpected but recoverable — rate limit hit, retry scheduled, fallback used	Always enabled; alert on >10/min
error	Failures requiring attention — external service down, DB error, unhandled exception	Always enabled; alert on >0 in critical paths
fatal	Process must exit — config invalid, port in use	Always enabled; immediate PagerDuty

22.3 Pino Configuration

Pino logger चा configuration app/logger.ts मध्ये आहे. खालील snippet essence दाखवते — async transport (Loki push), redact paths (PII masking), serializers (custom request/response logging), आणि child logger factory (per-module logger with module field pre-populated):

```
export const logger = pino({
  level: process.env.LOG_LEVEL || 'info',
  redact: {
```

```

paths: [
  'req.headers.authorization',
  'req.body.password',
  'req.body.aadhaar',
  'req.body.pan',
  '*.aadhaar',
  '*.pan',
  '*.password',
],
censor: '[REDACTED]',
},
serializers: {
  req: (req) => ({
    method: req.method,
    url: req.url,
    traceId: req.traceId,
    userId: req.user?.id,
    tenantId: req.user?.tenantId,
  }),
},
transport: process.env.NODE_ENV === 'production'
  ? { target: 'pino-loki', options: { host: process.env.LOKI_HOST } }
  : { target: 'pino-pretty' },
});

```

23. Observability

Observability हे 'system च interior state external outputs through infer करण्याची ability' आहे. PreOne मध्ये observability चे तीन pillars आहेत — Metrics, Logs, Traces. हे तीनही traceId ने correlate होतात — एक trace ID शी dashboard मध्ये सर्व data query केला जाऊ शकतो. यामुळे root cause analysis minutes मध्ये होते, नाहीतर hours.

23.1 Three Pillars

Pillar	Tool	What It Captures	Retention
Metrics	Prometheus + Grafana	RED (Rate, Errors, Duration) per endpoint; Saturation (DB pool, Redis memory); Utilisation (CPU, RAM, disk)	10s scrape interval

Pillar	Tool	What It Captures	Retention
Logs	Pino JSON → Loki	Structured; traceId correlation; 30-day hot retention + 1-year cold (S3)	Indexed by traceId, userId, tenantId
Traces	OpenTelemetry → Tempo	Distributed traces across services; span per DB/Redis/HTTP call	1% sampling in prod; 100% on errors
Alerts	Prometheus AlertManager → PagerDuty/Slack	Error rate, latency p95, saturation thresholds	Severity-based routing
Dashboards	Grafana	Per-module dashboard + system overview + tenant health	10s refresh
Synthetic Monitoring	Checkly / custom prober	Login, mark attendance, generate invoice every 5 min	Multi-region probes

23.2 Health Checks

Health checks k8s probes साठी critical आहेत — liveness probe determine करते की process restart करायचा का, readiness probe determine करते की traffic route करायचा का. खालील table सर्व health endpoints ची catalog करते:

Endpoint	What It Checks	Response	Used By
/health/live	Process is running	200 OK	Used by k8s liveness probe
/health/ready	Process can serve traffic (DB, Redis connected)	200 OK or 503	Used by k8s readiness probe
/health/startup	Process has finished booting	200 OK	Used by k8s startup probe
/health/deep	All downstream services reachable	200 OK with details	Manual + synthetic monitoring

23.3 Alerting Strategy

Alerts हे actionable असावे लागतात — जर alert fire झाला आणि on-call engineer ला काही करायचा नसेल तर alert noise आहे. PreOne चा alerting strategy strict आहे:

- Critical alerts (errors in critical paths, DB down, Redis down) → PagerDuty, page on-call immediately.
- Warning alerts (latency p95 > SLO, cache hit ratio < 80%) → Slack, page only if sustained for 5 min.
- Info alerts (deployment completed, backup finished) → Slack info channel, no page.
- Alerts include runbook link — on-call knows what to do without searching.
- Alert fatigue prevention: suppress known-issue alerts, group similar alerts, auto-resolve when issue clears.

24. Testing

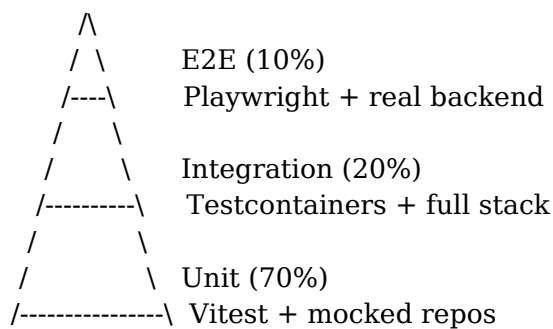
Testing हे software quality चा foundation आहे. PreOne चा testing approach classic test pyramid follow करतो — broad base of fast unit tests, narrow top of slow E2E tests. यामुळे feedback loop fast राहतो (unit tests ms मध्ये run होतात) आणि confidence high राहतो (E2E tests real user journeys cover करतात).

24.1 Test Pyramid

Layer	Tool	Scope	Coverage Target	Speed
Unit Tests (Domain)	Vitest	Pure TS — aggregate, entity, VO, spec, policy, domain service	100% coverage target	ms
Unit Tests (Application)	Vitest + mocked repos	Handler logic with mocked repositories + event bus	90% coverage target	ms
Repository Tests	Vitest + Testcontainers (Postgres)	Real DB — verify Prisma mapping + migrations	80% coverage target	seconds

Layer	Tool	Scope	Coverage Target	Speed
Controller Tests	Vitest + supertest	HTTP layer — DTO validation, auth guards, response shape	70% coverage target	ms
Integration Tests	Vitest + Testcontainers (full stack)	End-to-end within module — controller → handler → repo → DB	Critical paths only	seconds
Contract Tests	Pact	Consumer-driven contracts between bounded contexts	Per integration point	seconds
E2E Tests	Playwright + real backend	Full user journeys across modules	Smoke tests per release	minutes
Load Tests	k6	Performance benchmarks; capacity planning	Quarterly + pre-release	minutes
Chaos Tests	Litmus / Gremlin	Failure injection — DB kill, Redis flush, network partition	Quarterly	minutes

24.2 Test Pyramid Diagram



Speed: fast <-----> slow

Coverage: broad <-----> narrow

Cost: cheap <-----> expensive

24.3 Testing Rules

- 70% unit, 20% integration, 10% E2E — classic pyramid
- Domain tests must be pure — no DB, no network, no async IO
- Test files live next to source (co-location): student.aggregate.ts → student.aggregate.spec.ts
- Each PR must maintain or improve coverage — never decrease
- Tests run in CI on every push; failures block merge
- Flaky tests auto-disabled after 3 flakes; ticket created for fix
- Test data via factory functions (StudentFactory.create()) — never inline literals
- Mock at port boundary — mock repository interface, not Prisma client
- Integration tests use real Postgres via Testcontainers — never sqlite mock
- E2E tests run against staging with seeded data — reset between suites

24.4 Example — Aggregate Unit Test

Aggregate unit test pure domain logic verify करतो — कोणतेही DB, कोणतेही IO नाही. खालील example StudentAggregate चे behavior verify करतो. Tests co-located with source (student.aggregate.spec.ts). Vitest native ESM support ने fast execution मिळते:

```
describe('StudentAggregate', () => {
  describe('enroll', () => {
    it('should enroll student in section + raise StudentEnrolled event', () => {
      // Arrange
      const student = StudentFactory.create({ dateOfBirth: '2019-05-15' });
      const section = SectionFactory.create({ capacity: 30, enrolledCount: 15 });
      const academicYearId = UUID.v7();

      // Act
      const enrollment = student.enroll(section, academicYearId);

      // Assert
      expect(enrollment.studentId).toBe(student.id);
      expect(enrollment.sectionId).toBe(section.id);
      expect(section.enrolledCount).toBe(16);

      const events = student.pullEvents();
      expect(events).toHaveLength(1);
      expect(events[0]).toBeInstanceOf(StudentEnrolledEvent);
      expect(events[0].payload.studentId).toBe(student.id);
    });
  });
});
```

```

});

it('should throw BusinessException when section is full', () => {
  const student = StudentFactory.create();
  const section = SectionFactory.create({ capacity: 30, enrolledCount: 30 });

  expect(() => student.enroll(section, UUID.v7()))
    .toThrow(BusinessException);
  expect(student.pullEvents()).toHaveLength(0);
});
});
});

```

25. Performance

Performance हे user experience चा direct contributor आहे — slow app frustrates users आणि drives churn. PreOne चे SLO (Service Level Objective) निर्धारित आहेत आणि ते continuously monitor केले जातात. SLO breach alert trigger करते आणि performance optimization sprint मध्ये priority item बनते.

25.1 Performance Tactics

PreOne मध्ये 12 distinct performance tactics apply होतात. प्रत्येक tactic ची specific use case आहे — blanket application नाही. खालील table सर्व tactics ची catalog करते:

Tactic	Mechanism	Benefit	When
Connection Pooling	Prisma connection pool (default = $\text{num_cpus} \times 2 + 1$)	Tune per instance size	Avoid pool exhaustion under load
Read Replica Routing	Prisma client extension routes reads to replica	eventual consistency acceptable for reports	Writes always to primary
Batch Operations	Bulk insert/update via Prisma <code>createMany</code> / <code>updateMany</code>	Single round-trip	Limit to 500 rows per batch
Pagination	Cursor-based for large tables; offset for small	Cursor = stable sort key	Avoid OFFSET 10000+ — slow

Tactic	Mechanism	Benefit	When
Redis Cache	Hot reads (permissions, menus, KPIs)	≥ 90% hit ratio target	Invalidation via version bump
Async Jobs	Long-running work via BullMQ	5s+ operations offloaded	Prevents request timeout
Query Optimisation	EXPLAIN ANALYZE in CI on slow queries	Alert on > 100ms p95	Index review quarterly
N+1 Prevention	Prisma includes / DataLoader for nested reads	Lint rule: no nested find in loop	Auto-detect via Prisma logging
Field Selection	Prisma select() to fetch only needed columns	Reduces payload + memory	Especially for list endpoints
HTTP/2 + Brotli	Compression + multiplexing	30% smaller payloads vs gzip	Enabled at reverse proxy
CDN for Static	CloudFront for S3-hosted assets	1ms TTFB globally	Pre-signed URLs still valid through CDN
Database Indexing	Composite indexes for hot queries	Audit quarterly	Index-only scans where possible

25.2 Service Level Objectives (SLOs)

SLOs measurable performance targets आहेत. या targets consistently miss झाल्यास capacity planning किंवा architecture review triggered होतो. PreOne चे SLOs aggressive पण achievable आहेत — industry benchmarks शी aligned:

Metric	Target	Scope
API p50 latency	< 200 ms	99% of endpoints
API p95 latency	< 500 ms	99% of endpoints
API p99 latency	< 2000 ms	99% of endpoints
WebSocket message delivery	< 100 ms	99% of messages
Background job pickup	< 30 s	99% of jobs
Report generation (small)	< 30 s	Single branch, single term
Report generation (large)	< 5 min	Multi-branch, full year

Metric	Target	Scope
Database query p95	< 100 ms	Alert if exceeded
Cache hit ratio (permissions)	≥ 90%	Critical for auth path
Uptime	≥ 99.9%	Monthly — ~43 min downtime allowed

25.3 Connection Pooling

Prisma connection pool हा performance चा critical factor आहे — कमी pool size = requests queue, जास्त pool size = DB overload. PreOne ची pool sizing formula खालीलप्रमाणे आहे. हे formula AWS RDS max_connections limit शी aligned आहे — खूप जास्त connections असल्यास RDS throttle करते:

Formula: $\text{pool_size} = (\text{num_cpus} * 2) + 1$

Example (t3.medium instance - 2 vCPU):
 $\text{pool_size} = (2 * 2) + 1 = 5$

Example (t3.large instance - 4 vCPU):
 $\text{pool_size} = (4 * 2) + 1 = 9$

If 4 app instances run, total DB connections = $4 * 5 = 20$
 RDS t3.medium max_connections ~ 90 (safe margin)

Monitoring:

- Prisma logs pool wait time
- Alert if wait > 100ms (pool exhaustion)
- Scale up instance OR add read replica

26. Coding Standards

Coding standards हे codebase consistency सुनिश्चित करतात — ज्यामुळे code read करणे, review करणे, आणि maintain करणे सोपे होते. PreOne चे standards strict पण pragmatic आहेत — developer productivity balance with code quality. ESLint + Prettier automated enforcement करतात; बाकी rules PR review मध्ये check होतात.

26.1 Standards Catalog

- TypeScript strict mode enabled — no implicit any, no unchecked index access
- All types explicit — no var, no function keyword (use const arrow)
- PascalCase for classes/interfaces/types; camelCase for functions/variables

- UPPER_SNAKE_CASE for constants; kebab-case for filenames
- One class per file; filename matches class name (student.aggregate.ts → StudentAggregate)
- Import order: Node built-ins → external → internal absolute → relative → types
- No magic numbers — extract to named constant
- No any — use unknown + type guard if truly unknown
- Functional patterns preferred — pure functions, immutable data, no side effects
- Use Result<T, E> type for fallible operations (no thrown exceptions in domain layer)
- Comments explain WHY, not WHAT — code explains WHAT
- JSDoc on every public API (controller, service, repository method)
- No console.log — use Pino logger
- No Date — use ISO-8601 string or custom DateTime VO
- No Math.random for IDs — use UUID v7 (time-ordered)
- No setTimeout for business logic — use BullMQ delayed jobs
- No direct env access — use ConfigService
- No new Date().getFullYear() — use injected Clock service for testability

26.2 Pre-commit Hooks

Quality enforcement local level पर होण्यासाठी Husky pre-commit hooks install आहेत. प्रत्येक commit वर lint + format + type-check run होतात — failures commit block करतात. यामुळे broken code repository मध्ये push होत नाही:

```
// package.json
"husky": {
  "hooks": {
    "pre-commit": "lint-staged",
    "pre-push": "npm run test:unit"
  }
},
"lint-staged": {
  "*.ts": [
    "eslint --fix",
    "prettier --write",
    "tsc --noEmit"
  ]
}
```

26.3 PR Review Checklist

PR review मध्ये खालील checklist follow केला जातो. हे checklist reviewer ची cognitive load कमी करते — सर्व aspects systematically cover होतात:

- Domain logic in domain/ layer only — no business rules in controllers/handlers.
- Aggregate invariants enforced — no direct entity mutation outside aggregate.
- Repository returns domain aggregate — not Prisma model.
- DTOs have validation decorators + Zod schema + Swagger metadata.
- No console.log — use Pino logger.
- No any — use unknown + type guard.
- Tests added for new behavior — unit tests mandatory, integration tests for cross-module.
- Error codes are stable + documented in error catalog.
- PII not logged — verify redact paths cover new fields.
- New env vars documented in .env.example + secrets manager.

27. Deployment Readiness

Deployment readiness checklist हे production release च्या quality gate आहे — यातील कोणताही item pending असेल तर deployment block होतो. Checklist ADR-028 मध्ये defined आहे आणि release manager च्या approval नंतर production deployment proceed होतो. ही discipline production incidents कमी करते.

27.1 Pre-Deployment Checklist

Aspect	Item	Tool
Code	All PRs merged to main; no unmerged feature branches	Git
Migrations	Prisma migrations applied to staging; verified	prisma migrate deploy
Tests	All tests green in CI; coverage $\geq 80\%$	Vitest + Codecov
Lint	ESLint + Prettier clean; 0 warnings	ESLint

Aspect	Item	Tool
Env Vars	All required env vars set in production secrets store	AWS Secrets Manager
Feature Flags	New features behind flags; default off	Flagsmith / custom
Database Backup	Pre-deployment backup taken + verified restorable	pg_dump + S3
Rollback Plan	Documented rollback steps; tested in staging	Runbook
Monitoring	Dashboards + alerts configured for new metrics	Grafana + AlertManager
Logs	New log fields indexed in Loki	Loki
Traces	New spans visible in Tempo	Tempo
Load Test	k6 load test on critical new endpoints; meets SLO	k6
Security Scan	Snyk + SonarCloud clean; no new vulnerabilities	Snyk + Sonar
Documentation	README + API docs updated; Swagger regenerated	Compodoc + Swagger
Runbook	Operational runbook for new module updated	Confluence
On-call	On-call rotation updated; new alerts mapped to team	PagerDuty
Stakeholder Comms	Release notes circulated; stakeholders notified	Email
Window	Deployment window scheduled (low-traffic: 11 PM IST)	Calendar

27.2 Deployment Topology

Production deployment topology खालीलप्रमाणे आहे. हे topology AWS hosted आहे — multi-AZ for HA, autoscaling for elasticity, आणि managed services for operational simplicity. खालील table सर्व infrastructure components ची catalog करते:

Component	Service	Notes
Reverse Proxy	Nginx / AWS ALB	TLS termination, HTTP/2, brotli compression
App Tier	Kubernetes (EKS) — min 3 replicas	HPA on CPU + custom metrics
Database	Amazon RDS PostgreSQL 16 Multi-AZ	Primary + 1 read replica
Cache	Amazon ElastiCache Redis 7.2 (cluster mode off)	Multi-AZ, automatic failover
Queue	Same ElastiCache Redis (logical DB separation)	BullMQ uses DB 0; cache uses DB 1
Object Storage	Amazon S3	Server-side encryption (SSE-KMS)
CDN	CloudFront	For static assets + pre-signed S3 URLs
Secrets	AWS Secrets Manager	Auto-rotation for DB credentials
CI/CD	GitHub Actions → ECR → EKS	Blue-green for zero downtime
Observability	Grafana Cloud (Prometheus + Loki + Tempo)	Or self-hosted on dedicated node

27.3 CI/CD Pipeline

CI/CD pipeline GitHub Actions वर आहे — प्रत्येक push चा build + test + security scan होतो, main branch merge झाल्यावर staging deploy होतो, production deploy manual approval नंतर होतो. Blue-green deployment pattern zero-downtime releases सुनिश्चित करतो:

```

Push to feature branch
↓
CI: lint + typecheck + unit tests
↓ pass
CI: integration tests (Testcontainers)
↓ pass
CI: build Docker image + push to ECR
↓
CI: Snyk + SonarCloud scan
↓ pass
PR review + approval
↓ merge to main

```

```

CD: deploy to staging (auto)
↓
Smoke tests on staging
↓ pass
Manual approval (release manager)
↓
CD: blue-green deploy to production
↓
Health checks + smoke tests
↓ pass
Traffic switch + old version drain
↓
Release complete (rollback if anomalies)

```

28. Glossary

Backend terminology ची authoritative reference. नवीन terms introduce करताना या glossary मध्ये entry add करावी लागेल. Onboarding engineers साठी हा glossary पहिला read करायचा resource आहे — domain vocabulary build करण्यासाठी.

Term	Meaning
ACID	Atomicity, Consistency, Isolation, Durability — properties of reliable database transactions
ACL	Anti-Corruption Layer — translation layer between bounded contexts to prevent model pollution
ADR	Architecture Decision Record — short document capturing a single architectural decision
Aggregate	Cluster of domain objects treated as a single unit for data consistency; has one root
Bounded Context	Explicit boundary within which a domain model makes sense and terms have one meaning
BullMQ	Redis-backed queue library for Node.js; used for background jobs in PreOne
Casbin	Authorization library supporting RBAC, ABAC, ACL; used for PreOne permission checks

Term	Meaning
CQRS	Command Query Responsibility Segregation — separate models for reads vs writes
DLQ	Dead Letter Queue — queue for messages that failed processing after retries
DTO	Data Transfer Object — serialisable object for crossing process/network boundaries
DDD	Domain-Driven Design — software approach that mirrors business domain in code
HPA	Horizontal Pod Autoscaler — Kubernetes controller that scales pods based on metrics
Idempotency	Property where repeated operations produce same result — critical for safe retries
JWT	JSON Web Token — compact, signed token for stateless authentication
OTLP	OpenTelemetry Protocol — wire format for exporting traces, metrics, logs

Glossary (cont.)

Term	Meaning
Outbox Pattern	Writing events to DB table in same transaction as data; async publisher drains table
PII	Personally Identifiable Information — data that can identify an individual
Prisma	Type-safe ORM for Node.js / TypeScript — schema-first, generates client
RAG	Reduction in Force — but in PreOne context: Retrieval-Augmented Generation for AI features
RBAC	Role-Based Access Control — permissions assigned to roles; users inherit via role
Redis	In-memory key-value store — used for cache, pub/sub, queues, rate limiting

Term	Meaning
RLS	Row-Level Security — PostgreSQL feature that filters rows by user/tenant context
SAGA	Pattern for managing multi-step transactions with compensating actions on failure
SLO	Service Level Objective — target metric for service quality (e.g., 99.9% uptime)
Specification	Pattern encapsulating business rule as composable predicate (e.g., AgeEligibilitySpec)
Tenant	Customer organisation in multi-tenant SaaS — in PreOne, a school
UoW	Unit of Work — pattern that tracks changes and commits them as a single transaction
Value Object	Immutable object identified by its values (e.g., Address, Money) — no identity
Vitest	Modern test runner for Vite-powered projects — used for PreOne unit tests
WebSocket	Full-duplex communication protocol over TCP — used for real-time features in PreOne

29. Document Control

29.1 Version History

Version	Date	Author	Changes
1.0	2026-07-14	PreOne Architecture Team	Initial version — Backend Architecture Freeze

29.2 Approval Matrix

या document च्या freeze साठी खालील roles ची approval mandatory आहे. सर्व approvals recorded असल्यावरच document 'Architecture Freeze' status मध्ये move होतो:

Role	Name	Responsibility	Status
Chief Architect	[TBD]	Architecture review + sign-off	Pending
VP Engineering	[TBD]	Engineering approval	Pending
Tech Lead - Identity	[TBD]	Identity module review	Pending
Tech Lead - Finance	[TBD]	Finance module review	Pending
DevOps Lead	[TBD]	Deployment + observability review	Pending
Security Officer	[TBD]	Security + compliance review	Pending

29.3 Review Cadence

- Architecture Freeze: quarterly review (Q1, Q2, Q3, Q4)
- Major version bump (v2.0): on architectural shift (e.g., microservices split)
- Minor version bump (v1.1): on new chapter / significant section addition
- Patch version bump (v1.0.1): on clarification / typo fixes only

29.4 Distribution List

हा document खालील teams पर distributed आहे. Internal Engineering Reference — confidential, not for external distribution:

- PreOne Architecture Team (primary authors + maintainers)
- PreOne Engineering Team (all backend engineers)
- PreOne QA Team (test strategy + integration testing reference)
- PreOne DevOps / SRE Team (deployment + observability reference)
- PreOne Product Team (technical constraints awareness)
- PreOne Onboarding Team (new hire orientation reference)
- PreOne Security Team (security + compliance review)
- PreOne Leadership (architectural investment decisions)

29.5 Companion Documents

हा document खालील documents सह read केला जावा — त्यांचा context this document ला supplement करतो:

Document	Version	Relationship
Vision Document	v1.0	Strategic intent + non-functional requirements
BRC	v1.0	176 business rules — backend implementation reference
Master PRD	v1.0	Functional requirements — API endpoint mapping
DDD	v1.0	Domain model — backend module structure
ADR Catalog	v1.0	Architecture decisions — backend choices rationale
ERD v3.0	v3.0	Physical schema — repository data access layer
Prisma Schema v3.0	v3.0	Code-first schema — Prisma client generation
API Contract Catalog	v1.0	Endpoint contracts — controller implementation spec

29.6 Deliverables

या Backend Technical Design document मधून खालील production artifacts तयार होतील. हे artifacts implementation phase मध्ये engineering team द्वारे deliver केले जातील:

- Complete NestJS Folder Structure — 14 domain modules with consistent template.
- Controllers (~530 REST APIs) — versioned, Swagger-documented, RBAC-enforced.
- DTO Library — request/response/query DTOs with class-validator + Zod + Swagger.
- Application & Domain Services — workflow orchestration + pure business rules.
- CQRS Commands / Queries / Handlers — for complex workflows only (per ADR-012).
- Domain Event Catalog — 24 events with versioned schema + outbox publishing.

- BullMQ Job Definitions — 12 queues with retry policy + DLQ + alerting.
- Redis Cache Design — 10 cache layers with versioned invalidation.
- Transaction Strategy — aggregate = boundary, saga for multi-aggregate.
- Exception & Validation Framework — 6 validation layers + standardised error response.
- Logging & Observability Standards — structured logging + 3 pillars + alerting.
- Testing Strategy — test pyramid + coverage targets + Testcontainers.
- Backend Implementation Guidelines — coding standards + PR review checklist.